Fachhochschule Nordwestschweiz
Hochschule für Informatik

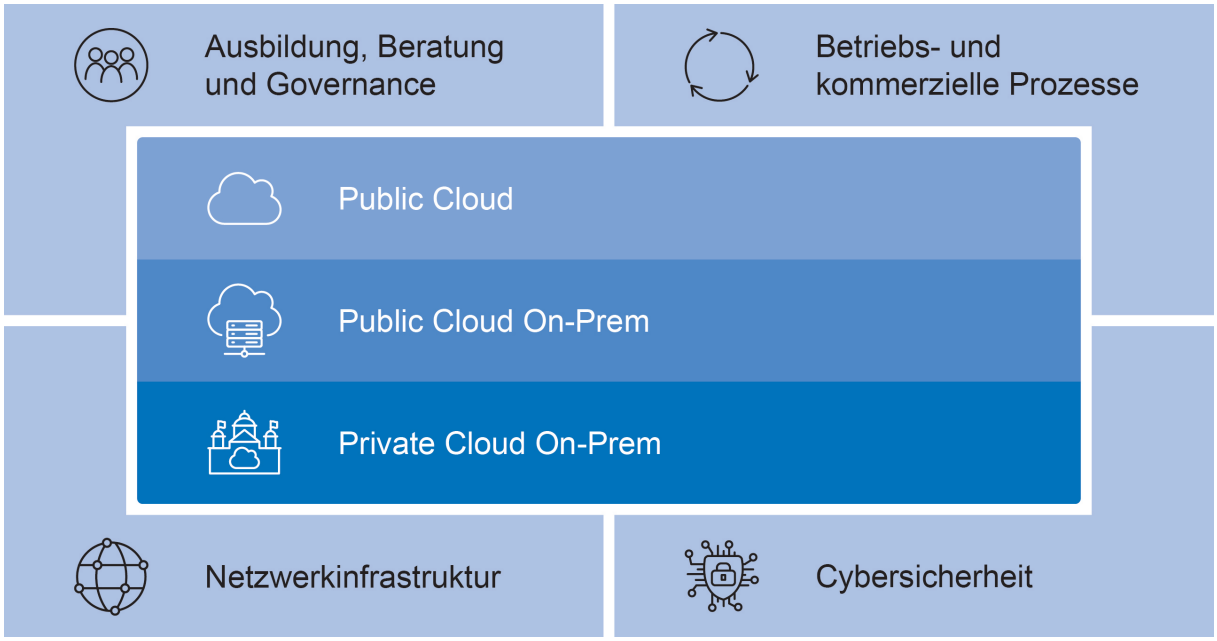
Schweizerische Eidgenossenschaft
Confédération suisse
Confederazione Svizzera
Confederaziun svizra

Eidgenössisches Finanzdepartement EFD
Bundesamt für Informatik
und Telekommunikation BIT

Cloud Compliance in einer multi-provider Cloud

Projektvereinbarung IP5

Windisch, März 2025



Studentin/Student	Frithjof Hoppe Benjamin Dätwyler
Expertin/Experte	Sebastian Matyas
Fachbetreuer/in	Sebastian Graf
Auftraggeberin	Thierry Perroud
Projektnummer	25FS_IMVS29

Fachhochschule Nordwestschweiz, Hochschule für Informatik

Abstract

In der vorliegenden Arbeit wird ein Konzept für die technische Umsetzung einer plattformübergreifenden Cloud-Compliance erarbeitet, wie sie im Rahmen des Swiss Government Cloud (SGC) Vorhabens des Bundesamt für Informatik und Telekommunikation (BIT) eingesetzt werden könnte. Das Konzept soll es ermöglichen, Compliance-Vorgaben in einer Multi-Cloud-Umgebung zu prüfen und durchzusetzen. Dabei wird ein Lösungskonzept gesucht, um eine zur Technologie agnostischen Implementierung von Compliance-Regeln zu ermöglichen. Die Arbeit umfasst die Analyse bestehender Ansätze, die Definition von Use-Cases sowie die Entwicklung eines Proof of concept (PoC)s und die Evaluierung des eingesetzten Lösungskonzeptes. Ziel ist es, eine robuste und benutzerfreundliche Lösung zu schaffen, die den Anforderungen der Bundesverwaltung (BV) gerecht wird und gleichzeitig die Integration verschiedener Cloud-Plattformen unterstützt.

Management Summary

Das vorliegende Dokument beschreibt die Entwicklung eines möglichen Lösungskonzeptes zur plattformübergreifenden Prüfung von Cloud-Compliance im Rahmen des SGC* Vorhabens des BIT*. Das Lösungskonzept basiert auf einem offenen Standard des National Institute of Standards and Technology (NIST), welcher es ermöglicht die Compliance über verschiedene Cloud-Plattformen hinweg zu verwalten. In einem PoC* wird gezeigt, wie dieses Lösungskonzept in der Praxis eingesetzt werden kann. Die Arbeit identifiziert wichtige Aspekte von Schutzobjekten in der Cloud und analysiert bestehende Konzepte zur Compliance-Prüfung. Das Ergebnis ist ein Lösungskonzept, das eine einfache Erweiterbarkeit und Integration in bestehende Systeme ermöglicht und gleichzeitig die Benutzerfreundlichkeit fördert. Die Evaluation des Lösungskonzeptes zeigt, dass es den Anforderungen der BV* entspricht und eine nachhaltige Lösung für die plattformübergreifende Cloud-Compliance darstellt.

Keywords:

Cloud, Hyperscaler, Compliance, Governance, Multi-Cloud, Open Security Controls und Assessment Language (OSCAL), CAC

Inhaltsverzeichnis

Abbildungsverzeichnis **v**

Tabellenverzeichnis **v**

1 Einleitung **1**

1.1 Zielsetzung 1

1.2 Ziele 2

1.3 Aufbau / Methodik 3

2 Compliance Aspekte **4**

2.1 Allgemeine Compliance Aspekte 4

3 Bestehende Ansätze **6**

3.1 Abbildung der Cloud-Ressourcen 6

3.2 Abbildung und Zusammenführen der Policies 9

3.3 Policy Check und Remediation 11

4 Evaluationskriterien des Lösungskonzepts **14**

4.1 Use-Cases 15

5 Lösungskonzept **16**

5.1 Entscheid Framework 16

5.2 Architektur 17

6 Proof of Concept **20**

7 Evaluation **27**

7.1 Zusammenfassung 30

8 Schlussbemerkungen **31**

8.1 Zielerreichung 31

8.2 Ausblick 31

9 Hilfsmittel **32**

Quellenverzeichnis **33**

Eigenständigkeitserklärung **36**

A Project Definition mit Azure Paths **37**

B Component Definition für Azure AKS **37**

C System Seucrity Plan für Azure Ressource Group mit AKS **39**

D Assessment Plan Results **44**

Abbildungsverzeichnis

1.1 Das Vorhaben SGC* im Überblick . . . 1

3.1 Teilprobleme und deren Zusammenhang . . . 6

3.2 Beispiel für OWL . . . 7

3.3 Summary of Core TOSCA Concepts from TOSCA . . . 7

3.4 Beispiel für TOSCA . . . 8

3.5 Exchange protocol from Agarwal et al. (2022) . . . 10

3.6 Layers und Models von OSCAL* nach NIST (2025) . . . 10

3.7 Generierten Beispiel für OSCAL . . . 12

3.8 Compliance-Checker mit Deployment-Model links und Compliance Regeln rechts nach Krieger et al. (2018) . . . 13

5.1 Deployment-View des Lösungskonzeptes . . . 16

5.2 Annahmen Konzepte bei verschiedenen Cloud-Providern . . . 17

5.3 Idealisierte Funktionsweise des Lösungskonzeptes . . . 19

6.1 Detaillierte Komponenten-Ansicht der PoC CAC Implementierung. Gestrichelte Komponenten/Verbindungen sind nicht Teil des PoCs bzw. nur theoretisch angedacht . . . 20

6.2 Reduziertes Klassenmodell vom implementierten PoC* . . . 21

6.3 Beispiel für ein `ProjectContext` mit `CloudPlattformPaths` für Azure . . . 22

6.4 Beispiel für eine Component Definition mit referenzierten Controls vom Si001 sowie Azure Policies . . . 23

6.5 Generierte Policy Initative in Azure für Control-ID t8.1 aus Si001 - IKT Grundschutz der Bundesverwaltung (Si001)* . . . 24

6.6 Beispiel für einen reduzierten Assessment Plan ob eine Ressource compliant ist . 25

6.7 Abfrage via Azure Python Software Development Kit (SDK)* für `RessourceId` und `PolicyAssignment` (Auszug aus assessment_service.py#L290) . . . 26

Tabellenverzeichnis

1.2 Ziele . . . 3

2.2 Kriterien zur technischen Bewertung der Frameworks . . . 4

4.2 Nicht-funktionale Anforderungen zur qualitativen Bewertung des Lösungskonzepts 14

4.4 Erwarteter Erfüllungsgrad der nicht-funktionale Anforderung (NFR)* für das Lösungskonzept . . . 15

5.1 Annahmen Konzepte bei verschiedenen Cloud-Providern . . . 17

7.1 Evaluation der Erfüllung der NFR* für das Lösungskonzept . . . 29

9.2 verwendete Hilfsmittel . . . 32

1 Einleitung

Mit dem Betreiben von Fachanwendungen in der Bundesverwaltung (BV) geht die Einhaltung von Compliance Regeln einher. Die Anforderungen an die Compliance für Fachanwendungen welche vom Bundesamt für Informatik und Telekommunikation (BIT) entwickelt und betrieben werden basieren auf verschiedenen Grundlagen. Diese Grundlagen können Standards, Richtlinien und Normen der BV* sein. Exemplarisch kann hier Si001* genannt werden. Momentan werden diese Anforderungen an die Compliance manuell und plattformspezifisch umgesetzt. Die Anforderungen selbst werden unterschiedlich ausgelegt und angewendet. Daraus resultiert unter anderem das Risiko, dass im Worstcase Security-Loopholes entstehen. Auch besteht das Risiko von einer Abweichung an den gewollten Zustand, wenn die Anforderungen an die Compliance nicht kontinuierlich überprüft werden.

Mit Ausblick auf das Swiss Government Cloud (SGC) Vorhaben gewinnt diese Situation besonders an Bedeutung, weil die Anzahl an Cloud-Plattformen in Benutzung in der BV* steigt und somit der Aufwand grösser wird eine einheitliche Anwendung der Regeln sicherzustellen. Das BIT* verfolgt mit dem SGC* Vorhaben die Transformation der Cloud-Infrastruktur der BV*. Das Vorhaben befindet sich momentan in der Konzeption. Die Vision sieht vor eine auf die Bedürfnisse des Bundes zugeschnittene hybride Multi-Cloud Infrastruktur aufzubauen wie in Abbildung 1.1 dargestellt wird.

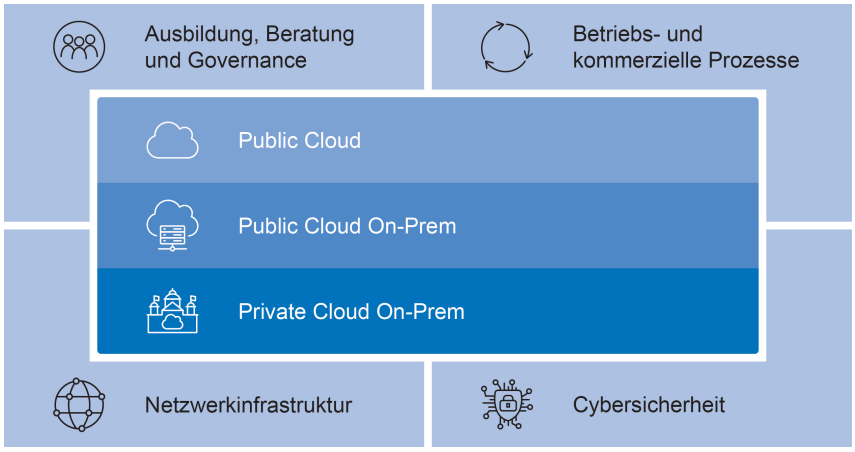


Abbildung 1.1: Das Vorhaben SGC* im Überblick

Die Ziel-Technologie/n für die drei Cloud Blöcke sind beim Schreiben dieser Arbeit noch nicht definiert worden bzw. sind Ergebnis von WTO-Ausschreibungen. Die Zielsetzung dieser Arbeit ist erstens ein Konzept für die technische Umsetzung einer plattformübergreifenden Cloud-Compliance zu definieren. Dieses Konzept soll theoretisch im Rahmen der SGC* angewendet werden können. Zweitens wird ein Konzept erstellt, mit welchem eine plattformübergreifende Cloud-Umgebung gegenüber Compliance-Vorgaben analysiert werden kann.

1.1 Zielsetzung

Für die Zielsetzung nehmen wir die Forschungsfragen aus der Projektvereinbarung [1] :

Welche wichtigen Aspekte eines Schutzobjektes in der SGC sind für die Compliance zu berücksichtigen?

[1] Die Formulierung wurde aus der Projektvereinbarung übernommen und lediglich grammatikalisch/syntaktisch geändert, um der restlichen Arbeit angeglichen zu sein

Die einzelnen Serviceangebote der Cloud-Provider bieten jeweils unterschiedliche Servicelevel an. Diese variieren teilweise zwischen Standorten (z.B. BIT onprem vs. Azure Zürich, AWS Dublin, etc.). Die wichtigsten Aspekte (von Schutzobjekten) welche in eine Umsetzung der Compliance gehören, sollen identifiziert werden.

Es soll erforscht werden, wie ein Mapping zwischen den relevanten Ausprägungen von Cloud-Services und den Aspekten von Schutzobjekten implementiert werden kann.

Welche Konzepte und etwaige Produkte existieren bereits zur plattformübergreifenden Prüfung und Enforcement von Compliance?

Im Rahmen der Lösungsfindung soll von bereits bestehenden Produkten und Ansätzen profitiert werden. Es soll herausgefunden werden, welche Produkte bereits existieren und wie diese die Compliance-Prüfung durchführen. Falls bereits bestehende Ansätze in der Bundesverwaltung (BV) existieren sollen diese vermerkt werden. Nützliche Ansätze und Konzepte sollen hervorgehoben und für eine allfällige Eigenimplementation ggf. in Betracht gezogen werden. Hier sollen Produkte untersucht werden, welche die Umsetzung der Compliance direkt/proprietär umsetzen, unabhängig von der dahinterliegenden Technologie.

Welche bestehenden Frameworks, Sprachen oder Konzepte sind am besten geeignet zur Umsetzung von Cloud-übergreifender Compliance?

Schutzobjekte (z.B. Applikationen) werden in der SGC cloud-native (via zugehörigen Deployment Code) deployt. Somit wäre der Deployment Code der richtige Ort festzulegen, welche Compliance-Regeln eingehalten werden müssen. Hier sollen Konzepte, Ansätze und Frameworks erforscht werden, welche die cloud-native Umsetzung der Compliance ermöglichen.

Wie muss ein Framework Compliance Regeln implementieren, sodass eine Anwendung durch verschiedenen Interessengruppen möglich ist?

Um verschiedene Regelwerke (z.B. Si001*) einheitlich anzuwenden ist ein (mensch- sowie maschinenlesbarer) Syntax notwendig, welche es erlaubt ebensolche Regelwerke zu definieren. Diese muss durch ein System interpretiert werden können damit die entsprechenden Massnahmen ergriffen werden können.

Es soll herausgefunden werden, wie ein solches Framework aussehen könnte und wie es verschiedene und unterschiedliche Regelwerke implementieren kann. Das Framework soll nicht die Regeln selbst enthalten, sondern lediglich die Möglichkeit bieten Regelwerke in einer Syntax zu formulieren.

Als Beispiel im Kontext der BV: ein Schutzobjekt muss z.B. nur die Si001* erfüllen während ein Anderes zusätzlich erhöhten Schutzbedarf (Eidgenossenschaft, 2023) aufweist und wieder ein Anderes gesetzlich zusätzlich ZertES (Eidgenossenschaft, 2016) einhalten muss.

1.2 Ziele

Die folgenden Ziele gelten für die gesamte Arbeit und orientieren sich an den Forschungsfragen. Es wird davon ausgegangen, dass die Erkenntnisse in einem Artefakt resultieren welches die genannte Ziele implementiert. Dieses Artefakt wird nachfolgend als Lösungskonzept bezeichnet, welches jedoch die Art des Artefakts nicht einschränken soll.

#	**Ziel**
1	Ein Konzept für die technische Umsetzung einer plattformübergreifenden Cloud-Compliance im Rahmen des SGC Vorhabens umgesetzt
2	Ein Framework welches mindestens die Ist-Situation einer multi plattformübergreifende Cloud-Umgebung gegenüber Compliance Vorgaben analysiert ist umgesetzt

Tabelle 1.2: Ziele

1.3 Aufbau / Methodik

Im Abschnitt 3 Bestehende Ansätze werden die Rechercheergebnisse zur Prüfung von Compliance in Teilaspekte zerlegt. Es werden die Gemeinsamkeiten von bestehenden Konzepten in Teilschritten identifiziert und auf mögliche Ergänzungen untereinander hingewiesen. Im Abschnitt Abschnitt 4 werden anschliessend auf Use-Cases basierenden Evaluationskriterien definiert. Durch die Use-Cases wird sichergestellt, dass der Output im Sinne der Stakeholder ist. Mit den Kriterien wird in Abschnitt 5 ein Lösungskonzept beschrieben. In 6 Proof of Concept erfolgt anschliessend eine Erprobung dieses Konzeptes im Rahmen eines PoC*. Zuletzt wird der PoC* mit den definierten Kriterien in Abschnitt 7 evaluiert.

2 Compliance Aspekte

Eine Anwendung in der Cloud kann verschiedene Anforderungen an die Compliance für die Plattform haben. Diese Anforderungen können sich auf die Plattform selbst beziehen, wie etwa die Einhaltung von Sicherheitsstandards. Sie können sich jedoch auch auf die Anwendungen beziehen, die auf der Plattform betrieben werden. Dieses Kapitel soll die Aspekte beleuchten, die für die Compliance in der Cloud prüfbar sein sollen. Dabei wird auf die bestehenden Standards und Richtlinien eingegangen, die für die Compliance relevant sind.

2.1 Allgemeine Compliance Aspekte

Verschiedene Standards und Richtlinien definieren ähnliche Aspekte oft in unterschiedlicher Form. Es wurde versucht, ähnliche Aspekte zusammenzufassen und zu vermerken in welchen Standards diese gefordert werden.

Kriterium	**Grobbeschrieb**	**Standards**
Geografischer und logischer Standort	Wo befinden sich die Daten und Anwendungen physisch, logisch und organisatorisch? Dies kann Auswirkungen auf die Einhaltung von Datenschutzgesetzen haben.	Si001*: I1 / 800-53r5: 3.11
SLA / Verfügbarkeit	Welche Verfügbarkeiten und Reaktionszeiten werden vom Plattformanbieter garantiert?	Si001*: I3, BSI-Standard 200-4: gesamt. Dok.
Zugriffskontrolle	Wie wird der Zugriff auf Ressourcen geregelt und überwacht?	Si001*: T5, T6.1, T8.1, Z2.2, Z3 / 800-53r5: 3.1, 3.4, 3.7, BSI-Standard 200-2: 7.3, Beispiele
Auditing und Monitoring	Welche Protokollierungs- und Überwachungsmechanismen sind vorhanden, um die Einhaltung von Compliance-Vorgaben zu überprüfen?	Si001*: Z5 / 800-53r5: 3.3, 3.4 / BSI-Standard 200-4: 5.2.1
Datenintegrität	Wie wird sichergestellt, dass die Daten während der Übertragung und Speicherung nicht verändert werden?	Si001*: I2, 800-53r5: 3.19
Backup und Wiederherstellung	Welche Strategien gibt es für die Datensicherung und -Wiederherstellung im Falle eines Ausfalls?	Si001*: I3

Tabelle 2.2: Kriterien zur technischen Bewertung der Frameworks

In Tabelle 2.2 wird sichtbar, dass es Überschneidungen von Si001*, NIST 800-53 Rev. 5 (Joint Task Force Interagency Working Group, 2020) sowie den BSI-Standards der 200er Serie (Bundesamt für Sicherheit in der Informationstechnik (2023) und Bundesamt für Sicherheit in der Informationstechnik (2017)) gibt. Wobei letztere eher Prozesse beschreiben anstatt technische Massnahmen, welche sich aus diesen ableiten.

Für die Aspekte werden oft andere Begriffe verwendet, bezwecken aber mehrheitlich dasselbe bzw. sind sich semantisch nahe. Betrachten wir beispielsweise den Aspekt der Zugriffskontrolle, so wird in Si001* auf die Aspekte T5, z2.2, t8.1, t6.1 und Z3 verwiesen. Diese Aspekte sind in

NIST 800-53 Rev. 5 unter den Punkten 3.1, 3.4 und 3.7 zu finden. Die genannten Aspekte eignen sich somit um in einem PoC* geprüft zu werden.

3 Bestehende Ansätze

Beginnend mit der Recherche von existierenden Konzepten / Ideen zur Prüfung der Compliance zeigt sich, dass weniger ganzheitliche Konzepte existieren, sondern viel mehr unterschiedliche Teilaspekte des Problems beleuchtet werden. Eine Zusammensetzung von diesen ermöglicht es einen allgemeinen Prozess zu definieren. Im Folgenden werden die identifizierten Teilaspekte und bestehenden Konzepte analysiert. Anschliessend fassen wir diese zusammen, um einen möglichen Aufbau für das BIT* zu definieren.

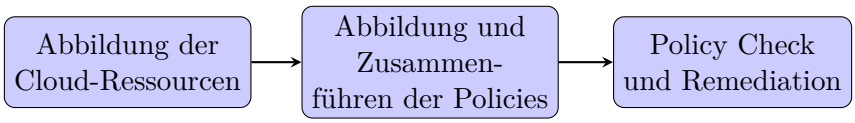


Abbildung 3.1: Teilprobleme und deren Zusammenhang

3.1 Abbildung der Cloud-Ressourcen

Vor der eigentlichen Analyse von Policies ist die Erfassung der Objekte, für welche man diese Analyse tätigen möchte, umso wichtiger. Der Begriff der Ontologie wird hierbei bei Wu (2015) aufgefasst als eine Möglichkeit die verwendete Struktur und die Ressourcen die diese Struktur ausmachen systematisch abzubilden. Es können hierbei pro Cloud-Plattform welche man analysiert verschiedene Ontologien existieren. Ziel ist Überlappungen zu identifizieren über welche man die Strukturen potenziell ineinander überführen kann.

Um diese abstrakte Idee der Ontologie maschinenlesbar abzubilden, verweisen Wu (2015) auf die Web Ontology Languge (OWL). Bei der OWL* handelt es sich gemäss OWL2 Spezifikation um eine semantische Web-Sprache, welche zur Repräsentation von Wissen über Objekte und deren Beziehungen untereinander dient. Diese stellt eine Erweiterung von Resource Description Framework (RDF) dar. Das folgende Snippet zeigt ein reduziertes Beispiel einer Ontologie, welche eine virtual Machine (VM) und einen angebundenen Speicher (hier `Disk`) definiert. Zu diesen Objekten ist ebenfalls eine Eigenschaft in Form von `hasDisk` definiert.

Nachdem die Struktur definiert wurde kann anschliessend eine konkrete Struktur abgebildet werden. Dies kann exemplarisch für ein Setup in einer Cloud-Plattform sein. Das folgende Beispiel zeigt die Instanzierung einer virtual Machine (VM) die anschliessend über die `hasDisk` mit einer `Disk`-Instanz in Beziehung gesetzt wird.

```
# Instance of the VM
ex:vm1 a ex:VMNode, ex:VMWithDisk ;
       ex:hasDisk ex:disk1 .

# Instance of the Disk
ex:disk1 a ex:Disk .
```

Reduziert man OWL* auf die wesentlichen Charakteristiken handelt es sich um eine auf Graphen basierte Darstellung.

Der Ansatz der Verwendung von Graphen zur Abbildung wird ebenfalls von TOSCA 2.0 aufgegriffen. Bei Topology and Orchestration Specification for Cloud Applications (TOSCA) handelt es sich um eine deklarative Sprache, welche basierend auf YAML erlaubt eine Service-Topologie mit eigens definierten Templates und Types zu definieren. Der Standard gibt lediglich das grundlegende Gerüst vor wie die einzelnen Komponenten definiert werden. Konkrete Definitionen sind dem Anwender der Spezifikation selbst überlassen.

```
@prefix ex: <http://example.org/cloud#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

ex:cloudOntology rdf:type owl:Ontology .

# Classes
ex:CloudResource rdf:type owl:Class .
ex:VMNode        rdf:type owl:Class .
ex:Disk          rdf:type owl:Class .

# Object properties
ex:hasDisk    rdf:type owl:ObjectProperty .

# VMWithDisk: a VMNode that has some Disk
ex:VMWithDisk a owl:Class ;
owl:intersectionOf (
    ex:VMNode
    [ a owl:Restriction ;
    owl:onProperty ex:hasDisk ;
    owl:someValuesFrom ex:Disk
    ]
) .
```

Abbildung 3.2: Beispiel für OWL

Der TOSCA* Standard von OASIS (Palma & Spatzier, 2013) greift diesen Ansatz der Modellierung von einer Ressourcenstruktur auf und kombiniert ihn mit der Orchestrierung.

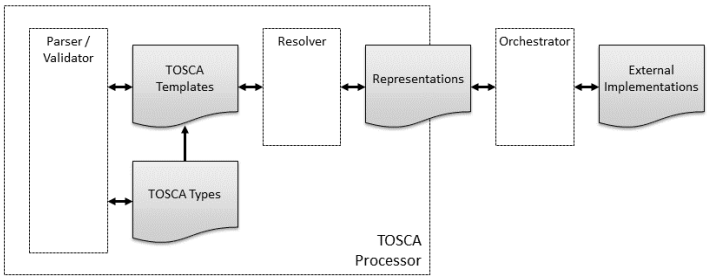


Abbildung 3.3: Summary of Core TOSCA Concepts from TOSCA

In Abbildung 3.3 werden die Beziehungen zwischen den Konstrukten von TOSCA* dargestellt. TOSCA* Templates definieren Services und deren Bestandteile. TOSCA* Types beschreiben die Values / Eingaben, welche für die Templates benötigt werden. Über den Resolver wird eine Represenation erzeugt. Wo External-Representations reale Artefakte wie Infrastruktur, K8s-Ressourcen etc. abbilden handelt es sich bei Representations um die modellierte agnostische Abbildung in TOSCA*. Als Orchestrator werden hierbei jegliche manuelle sowie automatisierte Prozesse betrachtet, welche den SOLL-Zustand wie er durch die Repräsentation beschrieben wird, in die External-Representation umwandeln. Beispiele hierfür wären Terraform oder

angebundene Provider wie Azure, Amazon Web Services (AWS) etc. Eine inhaltlich ähnliche Darstellung zu dem vorherigen OWL* Beispiel würde in TOSCA* v2.0 wie folgt aussehen:

```
tosca_definitions_version: tosca_2_0
description: Example TOSCA 2.0 Service Template - VM with disk and network
node_types:
    VirtualMachine:
        description: Virtual machine
        properties:
            osIdentifier:
                type: string
    Disk:
        description: Disk for a vm
        properties:
            size:
                type: integer
capability_type:
    Storage:
        description: Stores data
service_template:
    BasicVmOffering:
        inputs:
            operatingSystem:
                type: string
                description: Your offering name
        description: Construct of an vm with all its depending ressources
        node_templates:
            Vm:
                type: VirtualMachine
                description: Basic vm
                properties:
                    osIdentifier: {$get_input: operatingSystem}
                requirements:
                    -storage:
                        capability: Storage
                        node: Disk
            Disk:
                type: Disk
                    properties:
                        size: {$get_input: vmSize}
                capabilities:
                    storage:
                        type: Storage
```

Abbildung 3.4: Beispiel für TOSCA

Mit der Idee einer graphenbasierten Modellierung greift TOSCA* auch eine angelehnte Terminologie auf. Node-Types und zugehörigen Node-Templates stellen die Komponenten dar und werden über ein Service-Template und die Requirements in Beziehung zueinander gesetzt. Im vorherigen Beispiel werden erst für den Speicher und die VM* Nodes definiert und anschliessend über das Template `BasicVmOffering` in Beziehung gesetzt. Dies geschieht über das Requirement `Storage` im VM Template, welche auf das `Disk` Template verweist.

Borisova et al. (2020) zeigen exemplarisch, dass analog zu unserem VM Beispiel auch komplexere Konstrukte wie Artefakte in einem Kubernetes Setup mittels TOSCA* abgebildet werden können. Sie stellen aber gewisse Limitationen beim Mapping auf die im TOSCA 1.3 definierten Default-Types fest. Challita et al. (2020) identifizieren, dass mit TOSCA* zwar eine Modellie-

rung von Cloud-Ressourcen möglich ist, der kritische Aspekt vom Deployen aber nicht Teil der Spezifikation ist. Sie verwenden deshalb die Open Cloud Computing Interface (OCCI) Spezifikation nach Nyren et al. (2016), welche ein RESTful Protocol und API zum verwalten von Ressourcen definiert. Es wird gezeigt, dass durch das definieren eines Mappers ein TOSCA* model in ein OCCI* awares überführt werden kann. Die Anwendbarkeit wird durch zwei Studien gezeigt bei denen Sie WordPress und NodeCellar implementieren.

Zusammenfassend lässt sich feststellen, dass diverse deklarative Ansätze existieren welche es Erlauben Ressourcen sowohl auf Ebene der Infrastruktur als auch auf Anwendungsebene deklarativ zu modellieren. Die Umsetzung auf der jeweiligen Cloud-Plattforn wird hierbei losgelöst betrachtet.

3.2 Abbildung und Zusammenführen der Policies

Nach dem im vorherigen Abschnitt der Fokus auf der Abbildung von Ressourcen lag geht es in diesem Abschnitt um die Abbildung von Policies. Eine Policy wird hierbei von uns verstanden als *eine formelle Regel oder ein Satz von Regeln die definieren wie Cloud-Ressouren, Infrastrukturkomponenten und Services konfiguriert, bereitgestellt, genutzt und verwaltet werden dürfen.*

Li et al. (2014) identifizieren dass zwar grundlegende Sicherheitskonzepte über verschiedene Cloud-Plattformen hinweg erzwingbar sind, jeoch der Prozess von grober Sicherheitsanforderungen bis hin zur low-level Policy diverse Mappings erfordert. Für das in den vorherigen Abschnitten eingeführte Beispiel mit VM* und Disk können wir hier die bei NCSC, 2023, S.13 beschriebene Regel I3 verwenden welche sagt, dass eine Backup-Strategie dem Mehrgenerationen-Prinzip folgen soll und eine offline Speicherung wichtiger Informationen vorsehen muss. Nach Li et al. (2014) wird diese Anforderung in ein intermediate-level Model überführt, welches sowohl Aussage zur Syntax/Language als auch inhaltlich macht. Anforderungen aus unterschiedlichen Vorgaben müssen hierbei in ein intermediate-level Model zusammengeführt werden. In einem letzten Schritt wird anschliessend das Mapping auf die low-level Enforcementmechanismen der Cloud-Plattformen vorgenommen. Um korrespondierende Elemente zwischen Models zu identifizieren schlagen sie die Nutzung von semantischer Ähnlichkeit vor wie string level, linguistic level oder data level smiliarity.

Die Thematik der Modellierung wird bei NIST (2025) mit dem laufend weiterentwickelten Open Security Controls und Assessment Language (OSCAL) Standard aufgegriffen. Ziel ist es hier ein Compliance-Framework zu definieren welche eine maschinenlesbare Repräsentation von unter anderem Sicherheitsanforderungen in verbreiteten Formaten wie JSON/YAML ermöglicht. Agarwal et al. (2022) nutzen diesen Standard als Grundlage für die Formulierung eines Prozesses. Dieser Prozess nutzt die Maschinenlesbarkeit von OSCAL* um Sicherheitsanforderungen quasi direkt in ein Gesamtsystem integrieren zu können, was sie als compliance-as-code zusammenfassen. In diesem Prozess definieren sowohl Cloud-Anbieter und Security-Officers die Vorgaben wie auch Systeme die Erfüllung von Vorgaben in OSCAL*. Um dies zu erleichtern haben Agarwal et al. (2022) ein Tool namens Trestle [2] eingeführt, welches die bidirektionale Umwandlung von non-machine readable Policies (bspw. Markdown) zu Python Klassen und umgekehrt erlaubt.

Die Prüfung der Vorgaben wird, wie die auf Abbildung 3.5 sichtbar, durch die Integration von existierenden Produkten erreicht. Diese Komponenten werden als Policy Validation Points (PVP) bezeichnet und werden von den Cloud-Plattformen direkt oder als Connector zur Verfügung gestellt. Agarwal et al. (2022) legen hierbei verschiedene bestehende Produkte als Beispiele vor wie OpenSCAP [3], Tanium und Openshift Compliance Operator (OSCO).

[2] https://github.com/oscal-compass/compliance-trestle
[3] https://www.open-scap.org/

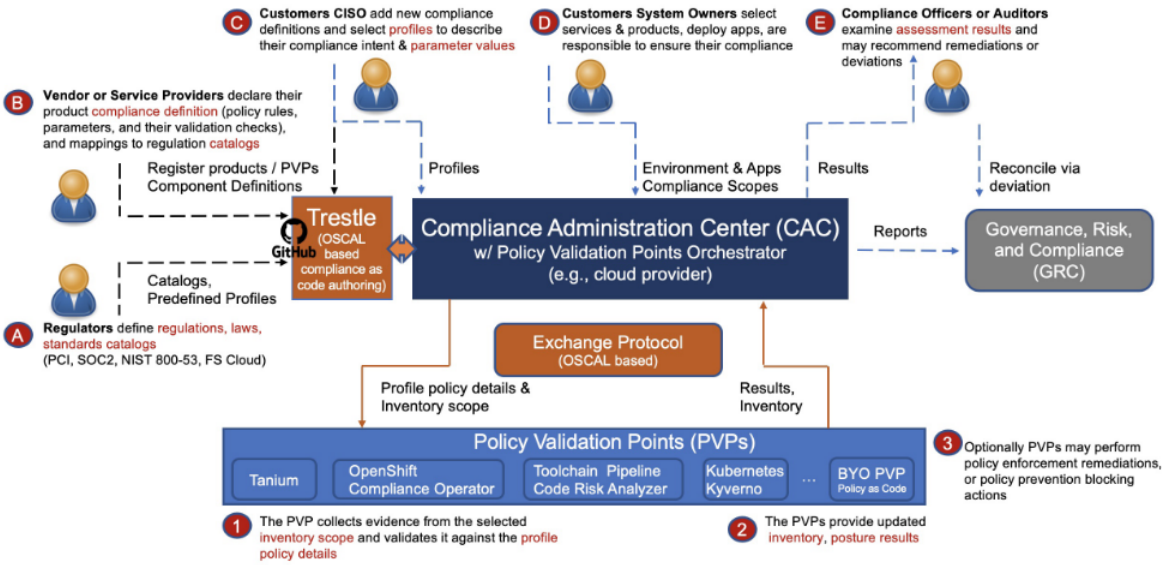


Abbildung 3.5: Exchange protocol from Agarwal et al. (2022)

OpenSCAP ist ein Framework welches den Security Content Automation Protocol (SCAP) der NIST* Standard implementiert. OpenSCAP spezialisiert sich auf Auditing und Remediation von Betriebssystemen und deren Konfigurationen. OpenSCAP eignet sich somit als Quelle für Assessment Results in OSCAL.

Wie die Zuweisung von den Vorgaben an die verschiedenen Komponenten erfolgt wird bei Krieger et al. (2018) betrachtet. Sie identifizieren, dass es für ein gegebenes deploytes System zwei Models gibt. Ein Compliance-Model welches bspw. von Security Experts in einem Compliance-Model-Repository verwaltet wird und ein Deployment-Model, welches analog von Experten und in separaten Repository verwaltet wird. Mittels des Compliance Repository werden Deployment-Compliance-Rules abgeleitet. Ein Compliance-Checker stellt auf Grund dieser Rules fest ob die Konstrukte im Deployment-Model-Repository und somit ein Deployment compliant ist. Es können nur Deployments erfasst werden können, welche mit dem Deployment-Compliance-Rules compliant sind. Ein Deployment-Experte muss somit nicht zwangsläufig Wissen über die Compliance besitzen um compliante Deployment-Models zu erstellen.

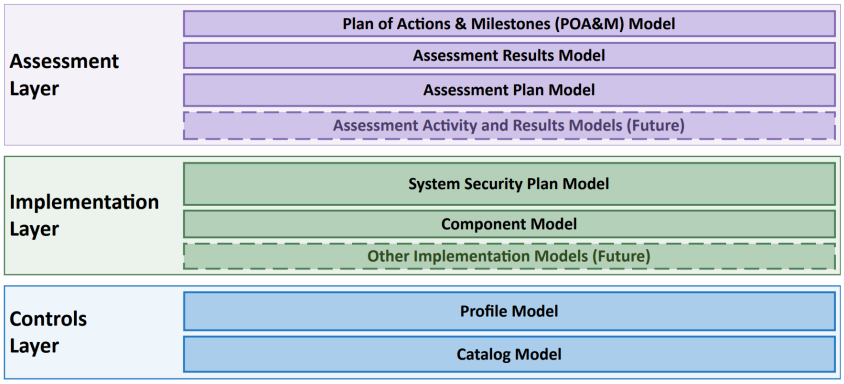


Abbildung 3.6: Layers und Models von OSCAL* nach NIST (2025)

Ein mögliche Darstellung des Mapping welches mittels der Deployment-Compliance-Rule erreicht werden möchte zeigt sich bei OSCAL*. NIST (2025) definiert in OSCAL* verschiedene Layers. Die Controls Layer in Abbildung 3.6 bildet die Sammlung an Compliance Regeln in welcher sich bspw. aus der Si001* Vorgabe von NCSC, 2023 befinden könnte. Catalog Models werden hierbei als eine Ansammlung an Regeln betrachtet welche in Subsets unterteilt werden können. Ein Beispiel von einem Subsets könnte hierbei eine Sicherheitsstufe sein wie sie bei NCSC, 2023, s.24 beschrieben wird. Diese Subsets werden in Profile Models abgebildet. Im aufbauendem Implementation Layer werden Controls definiert. Controls bilden die Bausteine, welche in einem System Security Plan Model (SSPM) zusammengefasst werden. Die zu implementierenden Rules sind den einzelnen Components angehängt.

In dem von uns eingeführten Praxis-Beispiel können die VM* und die Disk jeweils zwei Components darstellen welche in einem SSPM* zusammengefasst werden. Eine reduzierte Abbildung mittels OSCAL* könnte wie folgt aussehen. In diesem Beispiel wird die bereits erwähnte Regel I3 aus der Si001* nach NCSC, 2023, S.13 abgebildet. Die Regel macht Vorgaben entsprechend der Backup-Policy von Daten. Entsprechend wird bei der Component zur Disk der VM* mittels des Attribut `implemented-requirements` angegeben, dass die Disk diese Regel I3 implementiert.

3.3 Policy Check und Remediation

Mit den Ansätzen zu Mapping im vorherigen Abschnitt wurde die Beziehung hergestellt zwischen den Regeln und den Objekten / Komponenten auf welche dies Regeln angewendet werden. Krieger et al. (2018) und aufbauend Falazi et al. (2022) strukturieren die von ihnen eingeführten Deployment-Compliance-Rules in zwei Teile wie in Abbildung 3.8 ersichtlich wird. Detectors ermitteln ob Elemente in einem Deployment-Model eine Rule betreffen. Die Required-Structure wird danach genutzt um zu vergleichen, ob die von der Rule implizierte Struktur eingehalten wird. Falazi et al. (2022) schlagen vor das Konzept von Infrastructure as Code (IaC) zu nutzen um sowohl Deployment-Compliance-Rules wie auch Deployment-Models abzubilden. In Anerkennung der Heterogenität von bestehenden IaC* Technologien wird die Nutzung einer agnostischen Deployment-Model Language vorgeschlagen, welche als Angelpunkt dienen soll. Es können neue Deployment-Modelle mit ihr definiert werden und bestehende Deployments rekonstruiert/modelliert werden. Die Anwendung der Deployment-Compliance-Rule (Detectors und Descriptors) erfolgt nur auf dem agnostischen Modell und nicht auf den plattformspezifischen Modellen.

Agarwal et al. (2022) zeigen in Abbildung 3.5 wie Tools die Rules analysieren / anwenden in ein Gesamtsystem integriert werden können. Diese Tools könnten ihre Rule Analyse auf dem Detector/Structure-Konzept nach Krieger et al. (2018) aufbauen. NIST (2025) definieren mit OSCAL* im Assessment-Layer (siehe Abbildung 3.6) Models, mit denen die beiden vorher beschrieben Ansätze kombiniert werden könnten. Auf Grundlage eines SSPM* kann ein konkretes Assessment durchgeführt werden. Die Durchführung wird in einem Assessment Plan Model abgebildet und beschreibt neben dem *was* auch *womit* (Tools etc.) analysiert wird. Mögliche Tools werden bei Agarwal et al. (2022) mit ihren Policy Validation Points (PVP)s beschrieben. Durch die Verwendung von Tools ermöglicht sich neben dem manuellen auch ein kontinuierliches automatisiertes Assessment. Ein durchgeführtes Assessment resultiert in ein Assessment Result.

```
{
  "system-security-plan": {
    "uuid": "ssp-uuid-001",
    "metadata": {...},
    "system-characteristics": {
      "system-name": "Example App Infrastructure",
      "description": "Deployment composed of a VM and a storage disk."
      ...
    },
    "system-implementation": {
      "components": [
        {
          "type": "virtual-machine",
          "title": "Ubuntu Virtual Machine",
          "control-implementations": [
            {
              "uuid": "control-impl-vm-001",
              "source": "{link to e.g. Si001}",
              "implemented-requirements": [
                {
                  "uuid": "req-xx",
                  "control-id": "xxx",
                  "description": "{compliance rule in natural language}"
                }
              ]
            }
          ]
        },
        {
          "type": "storage",
          "title": "Geo-Redundant Disk",
          "control-implementations": [
            {
              "uuid": "control-impl-disk-001",
              "source": "source": "{link to e.g. Si001}",
              "implemented-requirements": [
                {
                  "uuid": "req-xxx",
                  "control-id": "xxx",
                  "description": "Geo-redundant storage .."
                },
                {
                  "uuid": "req-i-3",
                  "control-id": "I-3",
                  "description": "Automated backup policies .."
                }
              ]
            }
          ]
        }
      ]
    },
    "control-implementation": {...}
  }
}
```

Abbildung 3.7: Generierten Beispiel für OSCAL

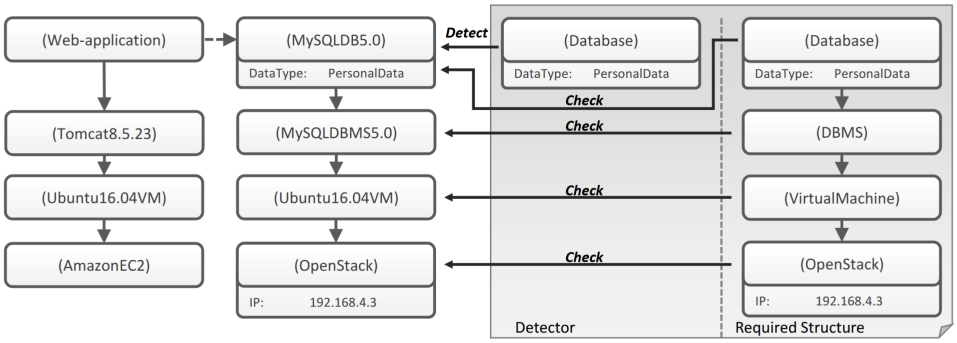


Abbildung 3.8: Compliance-Checker mit Deployment-Model links und Compliance Regeln rechts nach Krieger et al. (2018)

4 Evaluationskriterien des Lösungskonzepts

Anhand der Implementierung in einem PoC* soll ein Lösungskonzeptes evaluiert werden. Hierzu werden Use-Cases definiert, um ein Entwicklungsziel zu haben. Diese sollen hierbei möglichst Technologie-agnostisch sein, damit die Lösungsfindung nicht bereits zu Beginn eingeschränkt wird. Um die Umsetzung des jeweiligen Use-Case (UC) zu bewerten werden NFR*s definiert. Jeder UC* erhält vorgängig eine Zuweisung von NFR*, um zu definieren in welcher Qualität ein UC* erfüllt sein soll. Die Evaluation in Abschnitt 7 erfolgt anschliessend aufgrund der soeben beschriebenen Zuweisung.

#	NFR*	Metrik	Messung
K1	Interoperabilität	Lösungskonzept erlaubt die Integration von verschiedenen Plattformen bzw. besitzt keine plattformspezifischen Einschränkungen.	Anzahl der Plattformen die integriert werden können
K2	Integration	Lösungskonzept ermöglicht es sich in bestehende Systeme / Architekturen einzubetten und somit auf Bedürfnisse von Anwendern einzugehen.	Aufwand für Integration in bestehende Systeme in T-Shirt-Size*
K3	Robustheit	Lösungskonzept funktioniert trotz Anpassung der äusseren Umstände weiterhin. Gemeint ist, wie oft der Compliance Code angepasst werden muss bei Änderungen in der auditierten Anwendung oder dem System auf welchem die Auditierung dafür aufbaut.	Aufwand welche eine Änderung der äusseren Umstände (bspw. Anpassung Cloud-Provider) zu Folge hat mittels T-Shirt-Size*
K4	Dokumentation	Die dem Lösungskonzept zugrunde liegenden Komponenten sind ausreichend beschrieben für die Nachvollziehbarkeit von Konzepten und Nutzung durch Anwender	Geschätzter Grad an Auffindbarkeit von verschiedenen Features in der verfügbaren Dokumentation auf einer Skala von 1 bis 4, wobei 1 nicht auffindbar und 4 sofortiges auffinden bedeutet
K5	Benutzerfreundlichkeit	Lösungskonzept ist nur so kompliziert wie nötig und erlaubt es Anwendern (Entwicklern wie anderen Akteuren in den UC*) diese ohne zu steile Lernkurve zu verwenden.	Einarbeitungsaufwand in das Lösungskonzept und/oder darunterliegende Technologien in T-Shirt-Size*
K6	Erweiterbarkeit	Lösungskonzept bietet die Möglichkeit Eigenheiten von Cloud-Providern und deren Offerings abzubilden.	Geschätzter Grad der Erweiterbarkeit auf einer Skala von 1 bis 4, wobei 1 keine Erweiterung möglich und 4 spezifische Erweiterungen aufgrund des Cloud-Providers möglich bedeutet
K7	Automatisierung	Lösungskonzept bietet Möglichkeit, die Compliance-Prüfung zu versionieren und automatisieren.	Geschätzter Grad der möglichen Automatisierung auf einer Skala von 1 bis 4 wobei 1 keine sinnvolle Automatisierung möglich und 4 Automatisierung mit langfristigem Mehrwert möglich bedeutet

Tabelle 4.2: Nicht-funktionale Anforderungen zur qualitativen Bewertung des Lösungskonzepts

Der Erfüllungsgrad bzw. "Pass Wert" des Lösungskonzepts meint den Wert, bei dessen Erreichung das Lösungskonzept als sinnvoll erachtet wird, bzw. einen Mehrwert generiert. Der Wert

setzt sich sowohl aus den Beobachtungen im PoC*s, als auch aus Annahme einer potenziell zukünftigen Implementierung zusammen.

#	Pass Wert	Begründung
K1	> 1	Das Lösungskonzept muss zwangsläufig mehr als eine Plattform unterstützen, damit es effektiven Mehrwert generiert.
K2	L	Die Integration in eine bestehende Systemlandschaft ist häufig mit einem grösseren Aufwand verbunden, selbst wenn im Falle der SGC* diese aktuell noch nicht existiert.
K3	M	Die Robustheit hängt stark von dem Provider ab, auf den aufgebaut wird. Selbst wenn das Lösungskonzept stabil ist, kann die Abhängigkeit zu anderen Systemen (à la Azure oder AWS API) gleichwohl gewisse Anpassungen mit sich ziehen.
K4	2	Die Qualität der Dokumentation ist direkt relevant für die Anwendbarkeit des Lösungskonzeptes für die Stakeholder und deren Akzeptanz während der Anwendung.
K5	L	Das Lösungskonzept bringt wegen der Anwendbarkeit auf verschiedenen Systemen potenziell eine gewisse Abstraktion mit sich. Dies kann eine erhöhte Einstiegshürde beim Verständnis des Konzeptes bedeuteten.
K6	3	Die Erweiterbarkeit für bspw. built-in Features von Providern ist integral um davon im Lösungskonzept zu profitieren. Somit müssen nur jede Aufgaben im Lösungskonzept berücksichtigt werden welche nicht bereits durch Provider abgedeckt werden.
K7	2	Sinnvolle Automatisierung ist häufig nur beschränkt möglich. Mit 2 gibt es einen Mittelweg, der zwar eine reduzierte Automatisierung erwartet, aber somit eine die nicht laufend angepasst werden muss. Häufige Anpassungen der Automatisierung negiert häufig die Ersparnis, die sie hätte bringen sollen.

Tabelle 4.4: Erwarteter Erfüllungsgrad der NFR* für das Lösungskonzept

4.1 Use-Cases

Die UC* wurden aufgrund von Annahmen der bestehenden Arbeiten innerhalb des BIT* getroffen. Pro UC* werden ein oder mehrere User-Stories definiert, um eine konkrete Notwendigkeit aufzuweisen.

Übernahme von bestehenden Dokumenten

Wie einfach ist das Übersetzen von existierenden Richtliniendokumenten in das maschinenlesbare Format?
UC1: *"Ich als Compliance Officer will ich eine Si001 Richtlinie in das gewählte Format übersetzen für Andere zur Nutzung bereitstellen"*

Konformität von Anwendungen

Mit wie viel Aufwand müssen Anwendungsbetreiber rechnen, um Konformität zu beweisen, initial sowie fortlaufend?
UC2: *"Als Anwendungsbetreiber möchte ich meine zu deployende Anwendung klassifizieren können."*
UC3: *"Als Security Officer will ich prüfen, ob die Anwendung gemäss den von mir angeordneten Richtlinien läuft/betrieben wird."*

Konformität von Plattformen

UC4: *"Als Plattformbetreiber / -Owner will ich meine Plattform klassifizieren und die von der Plattform erfüllten Policies/Standards mappen und pflegen können."*

5 Lösungskonzept

Als Framework wird nachfolgende eine Ansammlung von Technologien oder Ansätzen verstanden, welche im Lösungskonzept aufgehen

5.1 Entscheid Framework

In Abschnitt 3 und Abschnitt 2 wurde die grobe Funktionsweise von bestehenden Ansätzen und Richtliniendokumenten analysiert. Die aus der Recherche gefasst Annahme ist, dass OSCAL* ein umfassendes und trotzdem einfach erweiterbares Framework für Abbildung von Artefakten für die Teilaspekte in Abbildung 3.1 ist. Zusätzliche zeichnet es sich durch die Förderung von NIST* aus, welches eine US-Behörde ist[4]. In der Vergangenheit hat sich gezeigt, dass Standard dieser Organisation häufig in der Industrie adoptiert werden. Zusammen mit W3C [5] und IETF[6] spielt NIST* eine wichtige Rolle bei IT-Standards. Die Annahme wäre somit, dass dieses Framework nachhaltig auf Rückhalt stossen wird.

Das Lösungskonzept baut des Weiteren auf dem Ansatz des Compliance Administration Center (CAC) auf, welches die Compliance über die verschiedenen Cloud-Plattformen und die zu prüfenden Ressourcen in unserem PoC* verwaltet. Das CAC* ist somit die Kernkomponente, welche die plattformspezifischen Richtlinien auf Grundlage der von den Benutzern bereitgestellten OSCAL*-Definitionen verwaltet. Dies können sowohl OSCAL Control (OSCAL) als auch konkrete Component Definitions (Component Definitions) sein. Das Prüfungsergebnis wird auf Grundlage dieser Richtlinien geliefert.

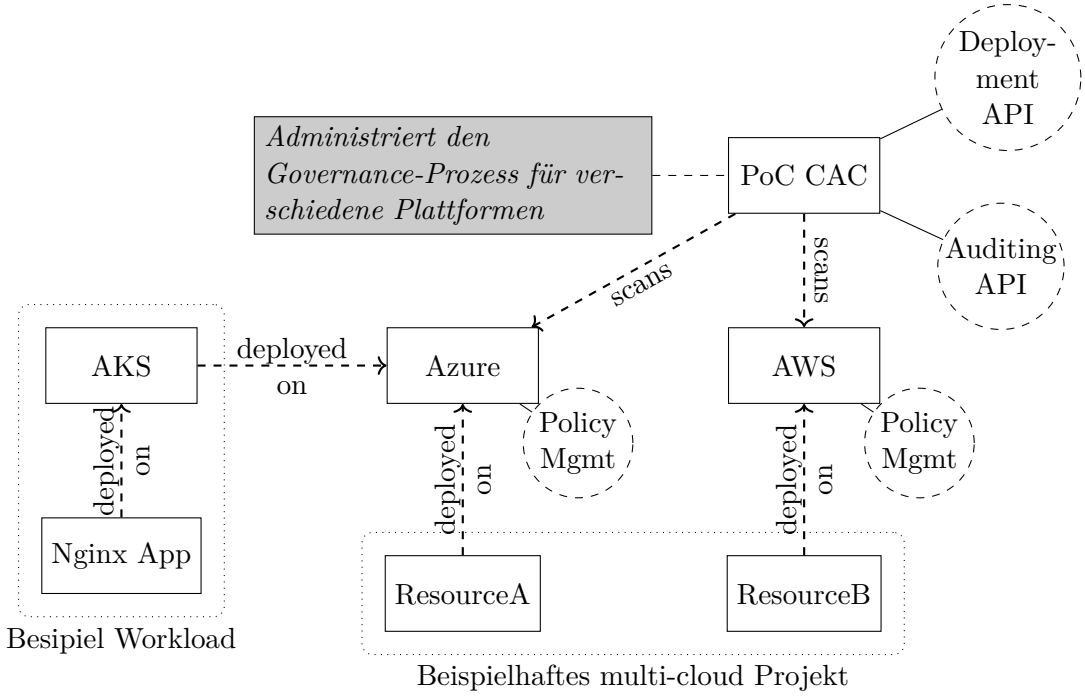


Abbildung 5.1: Deployment-View des Lösungskonzeptes

[4] US Departement of Commerce
[5] W3C
[6] IETF

5.2 Architektur

Die vorgeschlagene Architektur für das Löszungskonzept sowie die Details im Ablauf in Abbildung 5.3 basieren auf der Annahme, dass wesentliche Konzepte auf den verschiedenen Cloud-Plattformen repräsentiert sind. Die nachfolgenden Tabelle zeigt die in der Architektur verewendeten Konzepte

Konzept	Beschreibung	Äquivalent AWS	Äquivalent Azure
Policy	Regel zur Prüfung eines Subsets einer Control*	AWS Config Rules können prüfen, ob bestimmte Ressourcen konform mit gewissen Vorgaben sind[7]	Azure Policies können prüfen, ob bestimme Ressourcen, Groups, Subscriptions konform sind mit gewissen Vorgaben[8]
Policy Groups	Gruppierung von Policies, welche die Gesamtheit einer Control* abbilden	Conformance Packs sind eine Sammlung von Config Rules welche auf unterschiedliche Scopes angewendet werden können[9]	Policy Iniatives erlauben es verschiedene Policies zusammenzufassen und gemeinsam anzuwenden[10]
Scoping von Ressourcen	Ressourcen in einer Cloud können in virtuelle Einheiten unterteilt werden, sodass eine logische Gruppierung möglich ist, welche im Lösungskonzept als Project Scope bezeichnet wird. Der Scope hilft Policies eingeschränkt anzuwenden	Ressourcen können mittels Tags gruppiert und durch Resource Groups gesammelt angezeigt werden[11]	Subscriptions und Resource Groups[12] erlauben die Trennung in verschiedenen Ordnern inklusive Rechteverwaltung
Cloud-Plattform SDK	Komponente, welche den Compliance Status eines Scopes für eine gegebene Policy Group anzeigt	Boto3 erlaubt es mit der AWS API zu interagieren[13]	Verschiedene Packages erlauben die Interaktion mit Azure API. Wichtig für Policies ist das Package *Azure Mgmt Policy-insights*[14]

Tabelle 5.1: Annahmen Konzepte bei verschiedenen Cloud-Providern

Abbildung 5.2: Annahmen Konzepte bei verschiedenen Cloud-Providern

Analog zu AWS und Azure existieren bei Google Cloud Platform (GCP) z.B. *Organization Policies* und *Workload Manager*. Diese bieten ähnliche Möglichkeiten, um Compliance-Vorgaben zu definieren und durchzusetzen. AWS* bietet zusätzlich es *AWS Config*, *Security Hub*, *CloudTrail* und allenfalls *AWS IAM* sowie *AWS Organizations* als weitere Möglichkeiten zu Abbildung von Policies an.

Die verschiedenen Konzepte spielen wie folgt im Arbeitsablauf zuammen:

1. Der Benutzer stellt den Projektkontext bereit: Standorte oder Kennungen von z. B. Azure-Abonnements, AWS-Konten, Ressourcengruppen usw. sowie die Basisvorgaben hinsichtlich Vorschriften (z. B. vordefiniertes Profil Si001).

2. CAC analysiert die Bereitstellung und die verschiedenen darin bereitgestellten Ressourcen.
3. CAC ermittelt, ob bereits Component-Definitionen vorhanden sind (z.B. Si001).
4. CAC erstellt und implementiert Definitionen für den Projektkontext[15].
5. CAC überprüft regelmässig die zu prüfenden Ressourcen und erfasst deren Konformität.
6. Benutzer können zu verschiedenen Zeitpunkten nach dem Fortschritt der Prüfungskontrolle fragen bspw. ob die Konformität gegeben ist

[15] In diesem Vorschlag wird davon ausgegangen, dass das CAC Component-Definitionen (CDs) zumindest der Plattform erkennt und erstellt und diese mit der Bereitstellung verknüpft. CDs für die Anwendungen selbst müssten weiterhin vom Benutzer bereitgestellt werden

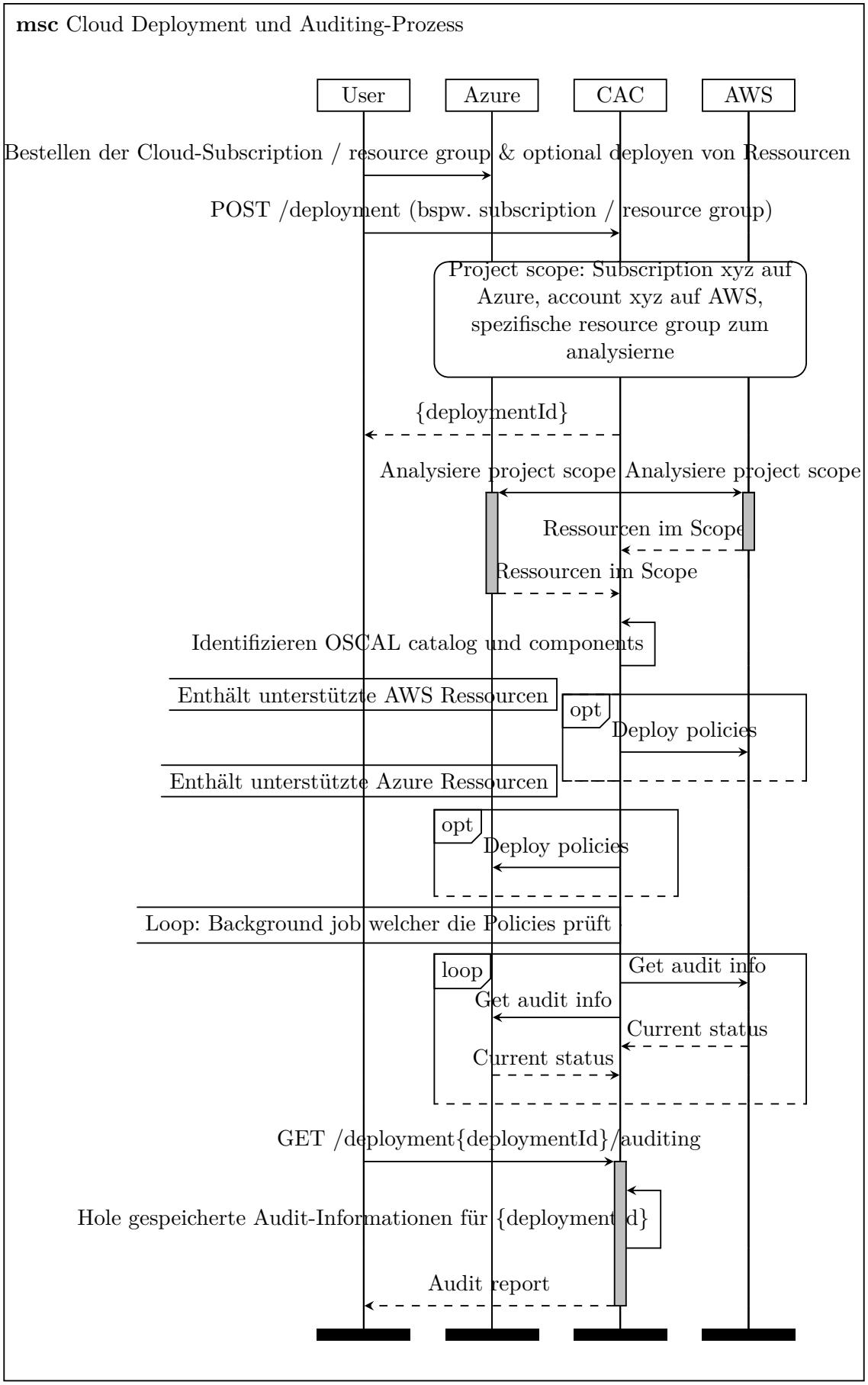


Abbildung 5.3: Idealisierte Funktionsweise des Lösungskonzeptes

6 Proof of Concept

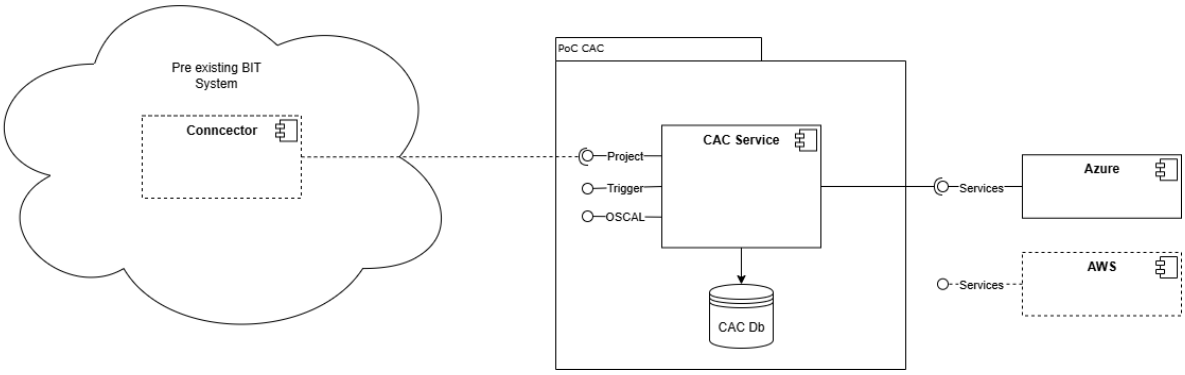


Abbildung 6.1: Detaillierte Komponenten-Ansicht der PoC CAC Implementierung. Gestrichelte Komponenten/Verbindungen sind nicht Teil des PoCs bzw. nur theoretisch angedacht

Im Rahmen der Erprobung des Lösungsansatzes wurde die "PoC* CAC*" Komponente umgesetzt wie sie in Abbildung 5.1 im Lösungsansatz angedacht wird. Das CAC* besteht aus zwei Komponenten: dem CAC* Service und der dazugehörigen Datenbank.

In der angedachten Situation für die SGC* wurde als Annahme getroffen, dass die Provisionierung von den Projekten selbstständig gemacht wird. Selbstständig bedeutet in diesem Kontext, dass es eine Marktleistung gibt mit der z.B. Azure Subscriptions oder AWS Accounts im Kontext des BIT* provisioniert werden können. Ressourcen welche jedoch auf diesen Subscriptions aufbauen müssen die Projekte eigenständig provisionieren. Als Projekt wird hierbei ein Vorhaben innerhalb des BIT* verstanden, welches Applikationen oder Ressourcen im Allgemeinen auf einer der für SGC* angebotenen Cloud-Provider laufen lässt. Der Connector wie er in Abbildung 6.1 angedacht wird und nicht Teil des PoC* ist soll als Integrator in diese bestehende Struktur im BIT* dienen. Konkret könnte dies heissen, dass den Projekten im BIT* als Teil der Provisionierungsmarktleistung ebenfalls eine weitere Marktleistung angeboten wird. Diese würde es den Projekten erlauben ihre Governance Bedürfnisse zu definieren, die sie für ihre Subscriptions / Accounts und die Ressourcen haben. Diese Bedürfnisse zusammen mit einer Baseline welche z.B. vom BIT* vorgegeben wird, würde dann die Governance Richtlinie darstellen, welche schlussendlich effektiv im CAC* für diesen Projekt-Kontext verwendet wird.

Das PoC* CAC* wurde in Python mittels FastAPI implementiert. FastAPI erlaubt es mit vergleichsweise (z.B. Spring Boot in Java) in kurzer Zeit eine Anwendung mit RESTful API zu erstellen. Die Wahl für Python ergab sich aufgrund der Nutzung durch die grösseren Hyperscaler. So wurden die Command Line Interface (CLI) zur Interaktion mit der Provisionierung auf den Plattformen sowohl bei AWS als auch Azure teilweise in Python implementiert[16]. Für die Persistierung der Daten wurde MongoDB als NoSQL Datenbank gewählt, weil die Modelle in OSCAL* häufig in JSON abgebildet werden. Eine dokumentenbasierte Datenbank wie MongoDB macht hierbei eine Abfrage einfacher als es z.B. bei einem relationalen PostgreSQL trotz JSONB[17] der Fall wäre. Der als Container deployte CAC* Service hat drei unterschiedliche Gruppierungen an Interfaces: Die Project-API über welche dem CAC* der Projekt-Kontext bekannt gemacht wird. Die OSCAL*-API über welche die jeweiligen generierten OSCAL* Inhalte zu einem gegebenen Projekt abrufbar sind. Die Trigger-API mit welcher manuell Aktionen ausgelöst werden können, welche für gewöhnlich periodisch ausgeführt werden würden. Die Anbindung an

[16] Azure CLI Implementierung-Details
[17] https://www.postgresql.org/docs/current/functions-json.html

Azure wurde durch die Nutzung der verschiedenen SDK* [18] erreicht. Die Abfragen welche im PoC* über die Azure SDK*s gemacht werden, sind mittels Service Principal[19] authentisiert, was einen anonymen bzw. nicht personenbezogenen Zugriff erlaubt. Architektonisch wurde bei der Implementierung zwar im Multi-Cloud-Kontext gedacht jedoch nur die Integration von Azure umgesetzt. Das heisst, die Implementierung ist zumindest theoretisch plattformagnostisch.

Strukturierung der Schnittstellen

Der CAC* strukturiert sich um das Domänenmodell herum, welches durch OSCAL* propagiert wird und im PoC* erprobt werden soll. Hierzu wurden die Python Modelle aufgrund des von OSCAL* bereitgestellten JSON-Schemas generiert. Zusätzliche Modelle wurden nur dort eingeführt, wo entweder keine Abbildung mit dem OSCAL* Domänenmodell möglich, oder dies nicht eindeutig abgeleitet werden konnte (z.B. aus der Dokumentation). In Abbildung 6.2 wird davon ausgegangen, dass der SSPM* der Ausgangspunkt bzw. der Übergang von vom Domänenmodell von OSCAL* in jenes in der PoC* Implementierung eingeführten CAC* ist.

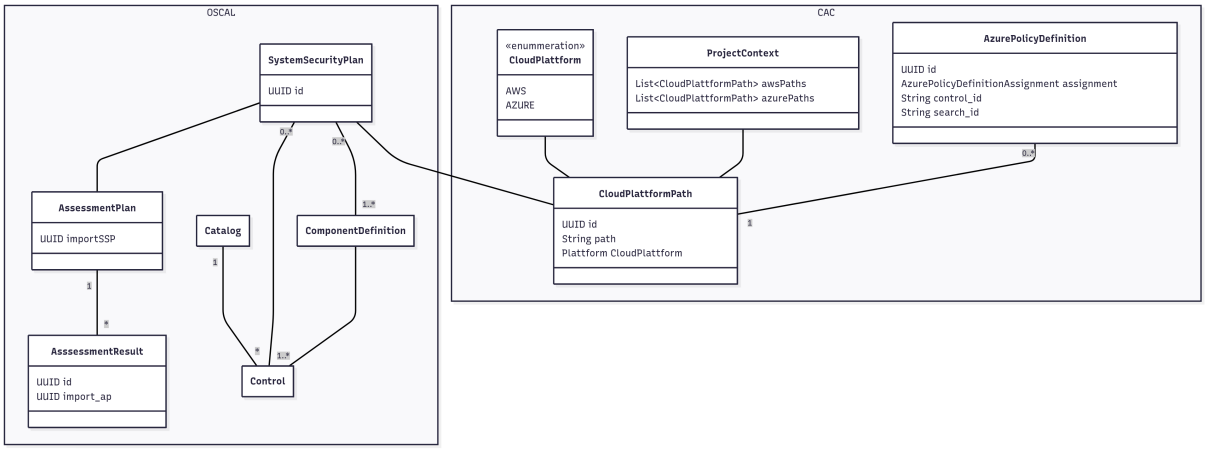


Abbildung 6.2: Reduziertes Klassenmodell vom implementierten PoC*

Der Anker in der CAC* Gruppierung ist das Modell des `CloudPlattformPath` bzw. des `ProjectContext`. Soll dem CAC* ein neues Projekt bekannt gemacht werden, wird über die Project-API der Kontext deklariert (bspw. durch die Connector Komponenten aus Abbildung 6.1). Mit Kontext ist die Gesamtheit der Referenzen in der jeweiligen Cloud gemeint wie sie in Abbildung 6.3 ersichtlich sind. Für Azure ist dies z.B. der Pfad einer Ressource-Gruppe oder Subscription. In AWS* gibt es entsprechende Äquivalente wie bereits in Unterabschnitt 5.2 gezeigt wurde.

[18] Azure Python SDK Overview
[19] Azure Service Principal

```
{
  "_id": {
    "$oid": "6895c4295a0c3aa88aef7aec"
  },
  "id": "143b64f3-8642-4653-a358-3a60dca87fa5",
  "name": "poc-resource",
  "aws_paths": [],
  "azure_paths": [
    {
      "id": "4f0587f9-5f31-4adb-9e37-25bea37db434",
      "path": "/subscriptions/abc/resourceGroups/ip5-scggov-sample",
      "plattform": "azure",
      "plattform_policy_reference": []
    },
    {
      "id": "b01faa50-088b-4cb8-a813-31232347c96f",
      "path": "/subscriptions/abc/resourceGroups/ip5-scggov",
      "plattform": "azure",
      "plattform_policy_reference": []
    }
  ]
}
```


Abbildung 6.3: Beispiel für ein `ProjectContext` mit `CloudPlattformPaths` für Azure

Das `id` Attribut, welches im `ProjectContext` Modell den einzelnen Referenzen zugewiesen wird, dient später dazu den Ursprung des Compliance Targets nachvollziehen zu können und wird auch `search_id` genannt. Die Referenzen löst der CAC* in einem weiteren Schritt bei den Cloud-Providern auf. In diesem Schritt werden die verschiedenen Ressourcen gesammelt und als Components in einem SSPM* gesammelt. Ein wichtiger Bestandteil dieses Prozesses ist die Verknüpfung mit einer etwaigen Component Definitions* und Controls. Im Fall vom PoC* dienen als Controls die einzelnen Abschnitte aus der Si001*, welche mit einer vordefinierten Component Definitions* für eine Azure Azure Kubernetes Service (AKS) Ressource zur Buildzeit des CAC* Service Contaninerimages hinzugefügt werden. Beim initialen Aufstarten des Services werden die Component Definitions*s in die Datenbank geladen, damit sie später in den anderen Modellen ebenfalls referenzierbar sind. Denkbar wäre an der Stelle auch eine Lösung mittels Versionierung im Sinne von GitOps, welche jedoch über den Rahmen des PoC*s hinausgeht. Für den AKS* wurden die Controls z2.2 für Zugriffe und t6.1 für Benutzerauthentifikation aus Si001* ausgewählt, für welches bereits existierende Azure Policies[20] verwendet werden konnten. Die Auswahl der Controls basiert auf Abschnitt 2. Das Matching wurde manuell gemacht könnte jedoch potenziell mittels Semantic Mapping gemacht werden, wie bereits in der Recherche bestehender Ansätze ersichtlich wurde. Beim Eingangs erwähnten Durchlaufen der Ressourcen wird geprüft, ob es eine passende Component Definitions* gibt. Bei einem Match werden die Controls in der Component Definitions* mit den referenzierten Polices in das SSPM* geladen. Das SSPM* enthält somit schlussendlich nicht nur eine Übersicht der Ressourcen, sondern ebenfalls welche Controls angewendet werden sollen und was dies für die einzelnen Cloud-Plattformen konkret bedeutet. Im Fall vom PoC* sind dies die Azure Policies ID wie in Abbildung 6.4 ersichtlich.

[20] Overview of Azure Policy - Microsoft Learn

```
{
  "component-definition": {
    "uuid": "e46eee83-a457-42d9-9cc4-96b00e64c638",
    "components": [
      {
        "uuid": "190eb8f2-0bd0-49b1-b378-8a2f072a1419",
        "type": "service",
        "title": "Azure Kubernetes Service (AKS)",
        "description": "Managed Kubernetes service in Microsoft Azure.",
        "props": [
          {
            "name":"azure-resource-type",
            "value":"Microsoft.ContainerService/managedClusters"
          }
        ],
        "responsible-roles": [
          {
            "role-id": "provider",
            "party-uuids": [
              "ee47836c-877c-4007-bbf3-c9d9bd805a9a"
            ]
          }
        ],
        "control-implementations": [
          {
            "uuid": "6c7e9b15-6d9c-4b61-a8a0-74ae51540efa",
            "source": "https://ip5-poc-webapp.azurewebsites.net/catalogs/si001.json",
            "description": "Si001 - IKT-Grundschutz in der Bundesverwaltung",
            "implemented-requirements": [
              {
                "uuid": "65e30b37-0640-4844-9f42-b2a7ae944bb1",
                "control-id": "t8.1",
                "description": "Cryptographic key management for AKS nodes.",
                "props": [
                  {
                    "name" : "azure-policy",
                    "remarks" : "Local authentication modes disabled only AD",
                    "value": "/providers/Microsoft.Authorization/policyDefinitions/
                    993c2fcd-2b29-49d2-9eb0-df2c3a730c32"
                  }
                ]
              }
            ]
          }
        ]
      }
    ]
  }
}
```


Abbildung 6.4: Beispiel für eine Component Definition mit referenzierten Controls vom Si001 sowie Azure Policies

Die Abbildung mit dem SSPM* (siehe Appendix C) erlaubt es nun einen Assessment Plan (AP) zu erstellen. Der AP* listet unter anderem die Tätigkeiten sowie die Organe / Entitäten

auf, welche diese durchführen. Im Rahmen des PoC*s ist dies ausschliesslich das CAC* selbst im Sinne eines technischen Nutzers. Aus dem Verknüpfen von den Policies und den jeweiligen Ressourcen-Gruppen / Subscriptions werden für Azure folgenden Tasks erstellt.

- `azure-deploy-initiative`: Wendet eine Azure Policy Initiative auf einen Scope an bzw. erstellt ein Policy Assignment
- `azure-check-initiative`: Prüft die Compliance eines Policy Assignments für den gegebenen Scope vom Assignment

Die Vorbedingung für den `azure-deploy-initiative` Task ist, dass bereits eine Policy-Initiative/Definition existiert, welche beim Bearbeiten dieses Tasks auf einen Scope angewendet werden kann. Deshalb werden beim Erzeugen des AP* bereits implizit aufgrund der Gesamtheit aller im SSPM* gefundenen Controls und Policies entsprechende Azure Policy Initiatives erstellt. Bei der Implementierung der Prüfung des `azure-check-initiative` Tasks hat sich gezeigt, dass eine Gruppierung der Policies in einer Policy Initiative nach Control-ID wie in Abbildung 6.5 diesen Vorgang vereinfacht.

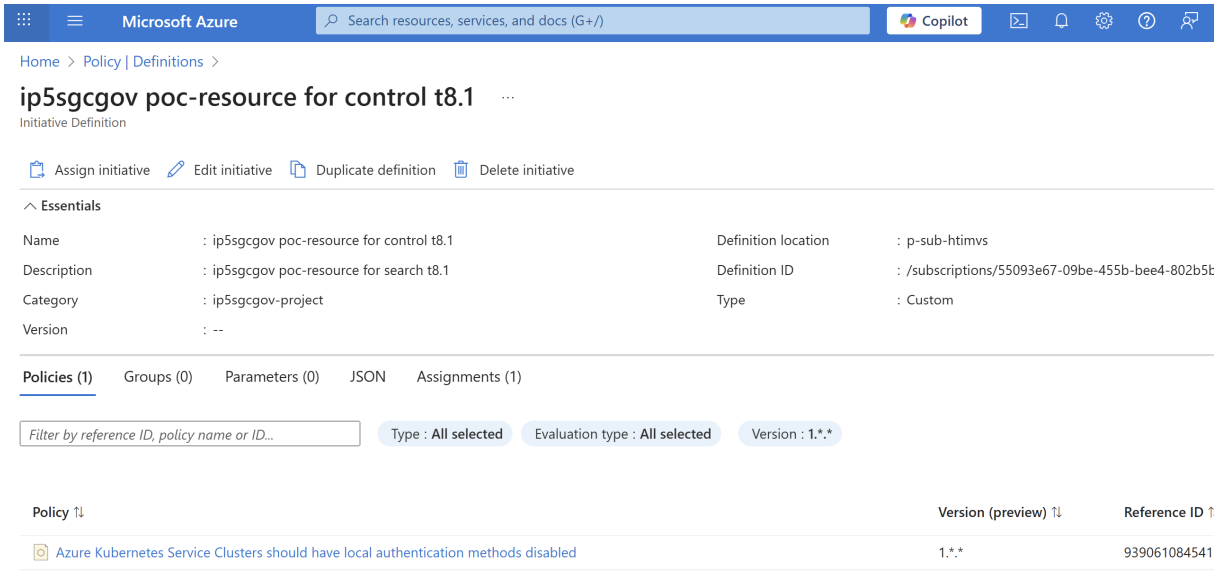


Abbildung 6.5: Generierte Policy Initative in Azure für Control-ID t8.1 aus Si001*

Der resultierende AP* enthält analog zum Beispiel in Abbildung 6.6 für alle Components, welche zu einer Policy gemappt wurden, einen Task. Diese Feingranularität erlaubt es wie bereits erwähnt die Compliance einer Component (im Beispiel vom PoC* der AKS*) gegenüber einer Control* zu bestimmen. In Abbildung 6.6 wird die Definition in Abbildung 6.5 mittels eines `azure-check-initiative` Tasks verknüpft. Mit den bis hierher erwähnten Konstrukten kann nach Erhalt der Validation-Results ein Assessment Plan Result (APR) erstellt werden. Hierzu werden zuerst die einzelnen Tasks durchlaufen.

```
{
  "_id": {
    "$oid": "6895c43d70814437a716d9e8"
  },
  "assessment-plan": {
    "uuid": "43064910-136e-47fd-9cda-90f57f383e69",
    "import-ssp": {
      "href": "9cebf0c6-e4d7-4693-96ec-005989c4b762"
    },
    "reviewed-controls": {
      "control-selections": [
        {
          "include-controls": [{"control-id": "t8.1"}]
        }
      ]
    },
    "tasks": [
      {
        "uuid": "064b2569-9d0a-4dd8-87cd-654c7194c178",
        "type": "action",
        "title": "Policy in initaitve are conformant",
        "description": "Ressource with component id xyz is
            conformant to control id t8.1",
        "props": [
          {
            "name": "azure-resource-id",
            "value": "/subscriptions/55093e67-09be-455b-bee4-802b5ba38768
            /resourceGroups/ip5-scggov/providers/
            Microsoft.ContainerService/managedClusters/sgc-cloudcompliance"
          },
          {"name": "cac-ap-task-type","value": "azure-check-initiative"},
          {"name": "oscal-control-id","value": "t8.1"},
          {
            "name": "azure-policy",
            "value": "/providers/Microsoft.Authorization/policyDefinitions/xyz"},
          {"name": "cac-project-serach-id","value": "123"},
          {"name": "oscal-component-id","value": "456"}
        ]
      }
    ]
  }
}
```


Abbildung 6.6: Beispiel für einen reduzierten Assessment Plan ob eine Ressource compliant ist

Im Rahmen des PoC*s wird dies über die Trigger-API ausgelöst, als Ersatz für einen Pipeline-Trigger oder einen periodischen Cron-Job. Die Ausführung geschieht im Azure Kontext durch die Nutzung des Policy Insight Clients, welcher eine Abfrage pro RessourceId und eine bestimmte PolicyAssignmentId erlaubt. Beide Informationen sind aus dem `azure-check-initiative` Task ableitbar.

```
policy_compliance = azure_service.get_policy_results_for_project(
    az_credential=credential,
    azure_resource_id=azure_resource_id,
    policy_definition=policy_definition,
    subscription_id=subscription_id,
)
non_compliant = any(
    r.compliance_state == "NonCompliant" for r in policy_compliance
)
logger.info(f"non_compliant is {non_compliant}")
include_controls.append(
    OscalCompleteOscalAssessmentCommonSelectControlById(
        control_id=policy_definition.control_id,
    )
)
reviewed_controls_id.append(control_id)

if non_compliant:
    findings.append(
        _create_fiding(
            state=OscalControlComplianceState.NON_COMPLIANT,
            component_id=component_id,
            description=f"Control {component_id} is 'not compliant'
            to control_id {control_id} in reguards of policy {azure_policy_id}",
            title=f"Control {component_id} is 'not compliant'
            to control_id {control_id} in reguards of policy {azure_policy_id}",
        )
    )
    assessment_log_entries.append(
        _create_assessment_log_entry(
            title=f"Analyzed ressource {component_id} is not compliant",
            description=f"Ressource with id {policy_definition.assignment.id}
            is not compliant to policy {policy_definition.assignment.id}",
        )
    )
```

Abbildung 6.7: Abfrage via Azure Python SDK* für `RessourceId` und `PolicyAssignment` (Auszug aus assessment_service.py#L290)

Die Resultate der Assessment werden als Findings hinterlegt. OSCAL* ist hier nicht eindeutig was die Abbildung anbelangt und bietet auch nicht direkt Beispiele an, an denen man sich orientieren kann. Die Erkenntnis aus Findings für die effektiven Ergebnisse und `assessment_log`s für einen chronologischen Log der Aktionen, die im Rahmen des Assessments durchgeführt wurden, sind jedoch für den PoC* ausreichend gewesen.

7 Evaluation

Die Evaluation des PoC*s gliedert sich in zwei Teile. Die Evaluation der definierten UC* in Unterabschnitt 4.1 und einer allgemeinen Beurteilung der NFR* über den einzelnen UC* hinweg.

UC1 Übersetzung von Richtlinien

Im Rahmen des PoC*s sollte die Auswahl an Control* z2.2, t8.1 und t6.1 auf entsprechende Policies in Azure gemapped und in OSCAL* abgebildet werden. Für z2.2 und t8.1 konnte dies mit geringem Aufwand erfolgen (siehe Appendix B). Das Mapping hatte sich hier wegen semantisch ähnlicher Formulierung der Controls im Si001* mit der Policy Beschreibung in Azure vereinfacht. Bei der Auswahl an Policies wurde sich im PoC* für die Kubernetes Ressource an den Azure Built-In Policies[21] orientiert. Die Controls in Azure bedurften zu dem keiner Parametrisierung. Parametrisierung bedeutet in diesem Fall, dass Policies für eine Anwendung auf einen Scope zusätzliche Werte benötigen. Für die Control t6.1 wurde für Azure initial die Policy "*Azure Kubernetes Service Clusters should enable Microsoft Entra ID integration*" gewählt, welche jedoch eine Parametrisierug braucht. Dies führte zur Erkenntnis, dass es eine zusätzliche Abstraktion für die Provider Integration braucht, welche solche Individualisierungen erlaubt. Für den Anwender (bspw. Compliance Officer) ist vor allem die Suche nach Policies zu den Controls ein arbeitsintensiver Task, welcher potenziell durch Automatisierung mittels semantischen Matching oder Künstliche Intelligenz (KI) vereinfacht werden könnte. Basierend auf diesem Mapping ist das Definieren von Component Definitions* für verfügbare Ressourcen mit reduzierten Aufwand möglich, wenn auch es sich um einen arbeitsintensiven Task handelt.

UC2 Klassifizieren von Inhalten

Eine Klassifizierung der Anwendung und insbesondere der Sensitivität des Systems wird durch das Lösungskonzept unterstützt. Die Beschreibung der System-Informationen in OSCAL* erlauben es out-of-the-box ein System, welches in einem SSPM* abgebildet wird, beliebig zu beschreiben und zu klassifizieren (siehe Appendix C). Ein Mapping auf die Ressourcen selbst wurde diesbezüglich zwar nicht im PoC* abgebildet, ist jedoch aufgrund der Möglichkeiten des Taggins in OSCAL* sowie AWS / Azure denkbar. Konkret können benutzerspezifische Tags verwendet werden, welche beim Provisionieren von Ressourcen durch zusätzliche Tagging-Policies enforced werden, um eine Klassifizierung durchzuführen. Diese könnte dann wiederum durch das Lösungskonzept auf den SSPM* übertragen und mit Policies aus einer Component Definitions* verknüpft werden. Für den Anwender (bspw. Anwendungsbetreiber) bedeutet dies, dass Tags als Metadaten Angaben beim Provisionieren von Ressourcen in dem "Environment under Governance" mitgegeben werden müssen.

UC3 Prüfen der Compliance

Eine Angabe über den Compliance Status konnte durch das Lösungskonzept für die Controls t8.1 und t6.1 generiert werden. Durch die Verwendung des APR* ist sowohl der aktuelle Zustand sowie die historischen Zustände sichtbar. Zusammen mit der Gruppierung der Policies nach Controls ist es dem Anwender (bspw. Security Officer wie Anwendungsbetreiber) möglich direkt aus dem APR* abzulesen welche Components compliant gegenüber welchen Controls sind (siehe Appendix D). Eine Einschränkung ist jedoch, dass das Framework nach einem blacklisting Approach arbeitet. Dies bedeutet, dass nur Ressourcen als non-compliant oder compliant interpretiert werden, für welche eine Component Definitions* definiert wurde.

UC4 Pflegen der Compliance Rules

Für den PoC* wurden die in OSCAL* gemappten Policies bzw. die Component Definitions* statisch definiert und können nicht sinnvoll durch einen Anwender (bspw Compliance Officer) im

[21] Azure Built-In Policies

Lösungskonzept verwaltet werden. Da es sich bei der Abbildung jedoch um ein JSON-Dokument handelt, ist ein handling mittels GitOps Ansatz bzw. durch ein Versionierungssystem denkbar. Kombiniert mit bereits verfügbaren Tools wie dem Oscal-Viewer[22] würde es eine einfache und effiziente Verwaltung erlauben.

#	Pass Wert	IST Wert	Begründung
K1	> 1	2	Die Architektur wie sie in Unterabschnitt 5.2 angedacht ist, zielt explizit darauf ab mehrere Plattformen zu unterstützen. Trotz den Unterschieden der betrachteten Plattformen (Azure und AWS) sind die Ansätze des Lösungskonzepts auf die verschiedenen Plattformen abbildbar (siehe Tabelle 5.1). Einem Plattformbetreiber ist es somit möglich eine eigene Plattform anzubinden.
K2	L	L	Die Projekt-API der Services ist so strukturiert, dass sie explizit OSCAL* agnostisch ist, da aus Sicht eines Anwendungsbetreibers i.d.R. irrelevant ist, wie die Abbildung der verschiedenen Bedürfnisse auf die Plattformen umgesetzt wird (siehe Appendix A). Bringt ein Anwendungsbetreiber jedoch eigene Anforderungen an Controls mit, welche zusätzlich abgebildet werden müssen, bedeutet dies möglicherweise einen Aufwand für den Plattformbetreiber oder Compliance Officer.
K3	M	S	Die SDK*s für Azure und AWS, welche im Lösungskonzept potenziell verwendet werden, haben stable Versionen welche jeweils einen Support von mindestens ca. 1 bis 2 Jahren haben. Entsprechend sind breaking Changes welche den Aufwand erhöhen würden unwahrscheinlich. Bzgl. der Entwicklung von OSCAL* selbst bedeutet eine Entwicklung des Standards nicht per se ein Zugzwang die bisherigen Abbildungen in OSCAL* zu aktualisieren, da keine Abfragen zu externen Komponenten in OSCAL* gemacht werden. Aus Sicht eines Anwenders welche Policies abbildet (wie ein Compliance Officer), wird das Lösungskonzept somit vermutlich als Robust wahrgenommen.

[22] OSCAL Viewer aus den https://oscal.io/tools/

#	Pass Wert	IST Wert	Begründung
K4	2	2	Die im PoC* für Azure verwendeten SDK*s waren zum Verwendungszeitpunkt nicht alle detailliert beschrieben. Trotz der verfügbaren Beispiele für gewisse Funktionalitäten wurden Grossteile der Verwendung der SDK* durch KI* abgeleitet. Für OSCAL* hat sich gezeigt, dass je höher man im Verlauf von Abbildung 3.5 gegangen ist, die Anzahl an Beispielen und Erläuterungen zum Domänenmodell abgenommen hat. Entsprechend wurde an jenen Stellen auf Annahmen aufgebaut, was jedoch die Kompatibilität zu anderen Implementationen basierend auf OSCAL* erschwert. Ein Beispiel hier ist die Abbildung der Compliance Status im APR*, wie im Appendix D gezeigt wird. Dies kann sich potenziell für den Compliance Officer oder Anwendungsbetreiber als Herausforderung darstellen, wenn es darum geht Artefakte von Dritten zu übernehmen (z.B. Policy Abbildung von Azure in OSCAL*).
K5	L	L	Aus Sicht des Compliance Officers ist eine Einarbeitung in den Aufbau von OSCAL* definitiv nötig und mit einem gewissen Aufwand verbunden. Konkret wird das Wissen benötigt wie man bspw. Component Definitions* für Control* erstellen kann. Zusätzlich muss für jede Plattform initial die Leistung erbracht werden Controls z.B. aus Si001* mit entsprechenden Policies von den verwendeten Plattformen zu verknüpfen. Anbieter wie Center for Internet Security (CIS) welche dies in für andere regulatorische Frameworks bereits getan haben können hier Abhilfe schaffen.
K6	3	2	Das Lösungskonzept baut aktuell auf der Annahme auf, dass wesentliche Konzepte auf den Plattformen existieren (siehe Unterabschnitt 5.2). Solange dies gegeben ist, ist sekundär, wie diese Konzepte in den entsprechenden plattformspezifischen Codesegementen umgesetzt sind. Die Erweiterung im Sinne eines Add-Ons für verschiedenen Plattformen scheint aus der aktuellen Implementierung im PoC* möglich. Um jedoch eine vollständige Add-On Fähigkeit zu haben wäre zusätzlich eine Schnittstellendefinition erforderlich, die es dem CAC* erlaubt bspw. plattformunabhängige Abfragen zu tätigen. Ein konkretes Beispiel ist hier die Parametrisierung von Policies auf Azure wie sie bereits bei der Evaluation UC1 erwähnt wurde.
K7	2	3	Wie bereits in der Evaluation für UC4 erwähnt ist eine Nutzung von GitOps bspw. für die Verwaltung von Component Definitions* ein gehbarer Ansatz. Zudem ist mit KI* ein kontinuierlicher Vergleich von den Policies in bspw Si001* und Controls denkbar um bei Anpassungen der plattformspezifischen Policies automatisch das Mapping in den OSCAL* Component Definitions* zu aktualisieren.

Tabelle 7.1: Evaluation der Erfüllung der NFR* für das Lösungskonzept

7.1 Zusammenfassung

Ingesamt ergibt sich ein durchaus positiver Eindruck aus der Evaluation. Die verschiedenen UC* konnten allesamt erfüllt werden, auch wenn z. T. von theoretischen Effekten bei einer Ausarbeitung des Lösungskonzeptes ausgegangen wurde. Hiermit ist gemeint, dass diese nicht direkt im reduzierten Umfang des PoC* gezeigt werden konnten. Anknüpfen tut hierbei auch der unterschrittene Pass-Wert für Extensibility. Dieser wurde gewählt weil im PoC* keine Add-On Erweiterung im Sinne von Plug and Play für weitere Plattformen erprobt werden konnte. Da jedoch konzeptionell ein potentieller Lösungsansatz, wie bereits bei der Evaluation beschrieben, existiert wird dieses Kriterium gleichwohl als erfüllt betrachtet.

8 Schlussbemerkungen

8.1 Zielerreichung

In dieser Arbeit wurden verschiedene Aspekte der Compliance in der Cloud untersucht und Überschneidungen zwischen den verschiedenen Standards und Richtlinien identifiziert.

Bei der Erforschung bestehender Ansätze wurden verschiedene technische Frameworks und Ansätze betrachtet aus denen OSCAL* hervorging. Mit OSCAL* wurden ein Ansatz gefunden, welcher nicht nur bereits Fuss gefasst hat durch die Standardisierung von NIST*, sondern auch einen Provider-agnostischen Ansatz verfolgt. Zusätzlich integriert sich OSCAL* gut in existierende Plattformen durch die Nutzung von bestehenden Prüfungsmechanismen wie Security Content Automation Protocol (SCAP) und Openshift Compliance Operator (OSCO).

In einem PoC* wurde versucht in einem beschränkten Umfang die Funktionsweise eines Lösungskonzeptes zu erproben, in welchen OSCAL* essenzieller Bestandteil war. Konkret wurde die Integration von OSCAL* in Azure Policies untersucht und ein Konzept entwickelt, wie Compliance-Aspekte in der Cloud geprüft werden können. Die Ergebnisse des PoC*s zeigen, dass die Integration von OSCAL* in Azure Policies möglich ist und dass die definierten Aspekte erfolgreich geprüft werden können. Das Compliance Resultat der konkreten Ressourcen war hierbei sekundär, lediglich die effektive Möglichkeit der automatisierten Prüfung der Aspekte war von Interesse.

8.2 Ausblick

Als Erweiterung des Lösungskonzeptes könnten weitere Cloud-Provider eingebunden werden, um dort die Integration mittels OSCAL* zu erproben. Proprietäre Mechanismen für Compliance und Auditing existieren in verschiedenen Cloud-Plattform wie bei Tabelle 5.1 für AWS und Azure gezeigt wurde. Für weitere Plattform wie GCP* wäre zu prüfen, ob diese Mechanismen ebenfalls mit dem Lösungskonzept kompatibel sind und wie sie in als PVP* integriert werden können.

Zusätzlich könnten weitere Aspekte der Plattform bzw. der darauf deployten Anwendungen validiert werden. Hierbei wäre zu prüfen, ob die jeweils zur Verfügung gestellten Aspekte der Cloud-Provider ausreichend sind oder ob zusätzliche Aspekte definiert werden müssen.

Das Problem der Compliance von anwendungsspezifischen Aspekten (wie z.B. ein Loginverfahren eines Frontends) wurde in dieser Arbeit ausgeklammert. Diese Problemstellung wird der Softwarearchitektur zugeordnet und die Prüfung deren ist nontrivial. Es könnte jedoch untersucht werden, ob und wie diese Aspekte in dem Lösungskonzept abgebildet werden können.

9 Hilfsmittel

Hilfsmittel	Verwendung	Betroffene Stellen
GitHub Copilot	Inspiration zum Aufbau & Darstellung des Kapitels	2 Compliance Aspekte
GitHub Copilot	Inspiration des Inhalts, Überarbeitung einzelner Textabschnitte	8 Schlussbemerkungen
DeepL	Übersetzung des initial Englisch geschriebenen Textes auf Deutsch	5 Lösungskonzept
ChatGPT	Generieren von Code Beispielen für die Nutzung der Azure SDKs	Implementierung in azure_service.py
ChatGPT	Generieren von Code Beispiel für OWL	Abbildung 3.2
ChatGPT	Generieren von Code Beispiel für TOSCA	Abbildung 3.4
ChatGPT	Generieren von OSCAL Beispielen	Abbildung 3.7
ChatGPT	Inspiration von NFR sowie Messkriterien	7.1 Evaluation
ChatGPT	Suche von Alternativen auf anderen Cloud Providern	Abbildung 5.2

Tabelle 9.2: verwendete Hilfsmittel

Quellenverzeichnis

Agarwal, V., Butler, C., Degenaro, L., Kumar, A., Sailer, A., & Steinder, G. (2022). Compliance-as-Code for Cybersecurity Automation in Hybrid Cloud [ISSN: 2159-6190]. *2022 IEEE 15th International Conference on Cloud Computing (CLOUD)*, 427–437. https://doi.org/10.1109/CLOUD55607.2022.00066

Borisova, A., Shvetcova, V., & Borisenko, O. (2020). Adaptation of the TOSCA standard model for the Kubernetes container environment [Conference Name: 2020 Ivannikov Memorial Workshop (IVMEM) ISBN: 9781728190884 Place: Orel, Russia Publisher: IEEE]. *2020 Ivannikov Memorial Workshop (IVMEM)*, 9–14. https://doi.org/10.1109/IVMEM51402.2020.00008

Bundesamt für Sicherheit in der Informationstechnik. (2017). *Informationssicherheit und IT-Grundschutz (E-Book): BSI-Standards 200-1, 200-2, 200-3* (3rd ed). Bundesanzeiger Verlag.

Bundesamt für Sicherheit in der Informationstechnik (Hrsg.). (2023). *Business Continuity Management: BSI-Standard 200-4*. Reguvis Fachmedien GmbH.

Challita, S., Korte, F., Erbel, J., Zalila, F., Grabowski, J., & Merle, P. (2020). Model-Based Cloud Resource Provisioning with TOSCA and OCCI. *ArXiv*. Verfügbar 1. April 2025 unter https://www.semanticscholar.org/paper/Model-Based-Cloud-Resource-Provisioning-with-TOSCA-Challita-Korte/20b6f353a04cdc34800c7feee8845a7038a6f5dd

Eidgenossenschaft, S. (2016). Bundesgesetz über Zertifizierungsdienste im Bereich der elektronischen Signatur und anderer Anwendungen digitaler Zertifikate. https://www.fedlex.admin.ch/eli/cc/2016/752/de

Eidgenossenschaft, S. (2023). *Erhöhter Schutz*. Verfügbar 12. März 2025 unter https://www.ncsc.admin.ch/ncsc/de/home/dokumentation/sicherheitsvorgaben-bund/sicherheitsverfahren/erhoehter-schutz.html

Falazi, G., Breitenbucher, U., Leymann, F., Stotzner, M., Ntentos, E., Zdun, U., Becker, M., & Heldwein, E. (2022). On Unifying the Compliance Management of Applications Based on IaC Automation. *2022 IEEE 19th International Conference on Software Architecture Companion (ICSA-C)*, 226–229. https://doi.org/10.1109/ICSA-C54293.2022.00050

Joint Task Force Interagency Working Group. (2020, 23. September). *Security and Privacy Controls for Information Systems and Organizations* (Edition: Revision 5). National Institute of Standards und Technology. https://doi.org/10.6028/NIST.SP.800-53r5

Krieger, C., Breitenbücher, U., Képes, K., & Leymann, F. (2018). An Approach to Automatically Check the Compliance of Declarative Deployment Models. https://www.iaas.uni-stuttgart.de/publications/INPROC-2018-42-An-Approach-to-Automatically-Check-the-Compliance-of-Declarative-Deployment-Models.pdf

Li, J., Ouedraogo, W. F., & Biennier, F. (2014). Multi-dimensional model driven policy generation. In M. Helfert, F. Desprez, D. Ferguson & F. Leymann (Hrsg.), *Cloud computing and services science* (S. 69–85). Springer International Publishing. https://doi.org/10.1007/978-3-319-11561-0_5

NCSC. (2023). SI001 - IT-Grundschutz in der Bundesverwaltung. Verfügbar 26. April 2024 unter https://www.ncsc.admin.ch/dam/ncsc/de/dokumente/dokumentation/vorgaben/sicherheit/si001/Si001-IT-Grundschutz_V5-1-d.pdf.download.pdf/Si001-IT-Grundschutz_V5-1-d.pdf

NIST. (2025). *OSCAL - Open Security Controls Assessment Language*. Verfügbar 23. März 2025 unter https://pages.nist.gov/OSCAL/

Nyren, R., Edmonds, A., Papaspyrou, A., Metsch, T., & Parak, B. (2016). Open cloud computing interface – core.

Palma, D., & Spatzier, T. (2013, 25. November). *Topology and Orchestration Specification for Cloud Applications Version 1.0* [TOSCA Standard]. Verfügbar 23. März 2025 unter https://docs.oasis-open.org/tosca/TOSCA/v1.0/os/TOSCA-v1.0-os.html#_Toc356403634 NOTE: Technische Spezifikation von TOSCA v1.0.

Wu, Z. (2015). Multi-cloud policy enforcement through semantic modeling and mapping [Conference Name: 2015 IEEE International Conference on Semantic Computing (ICSC) ISBN: 9781479979356 Place: Anaheim, CA, USA Publisher: IEEE]. *Proceedings of the 2015 IEEE 9th International Conference on Semantic Computing (IEEE ICSC 2015)*, 448–451. https://doi.org/10.1109/ICOSC.2015.7050849

Akronyme

AKS Azure Kubernetes Service. 22, 24
AP Assessment Plan. 23, 24
APR Assessment Plan Result. 24, 27, 29
AWS Amazon Web Services. 8, 17, 21

BIT Bundesamt für Informatik und Telekommunikation. ii, 1, 6, 15, 20
BV Bundesverwaltung. ii, 1

CAC Compliance Administration Center. 16, 20–22, 24, 29
CIS Center for Internet Security. 29
CLI Command Line Interface. 20
Component Definitions Component Definitions. 16, 22, 27, 29
Control OSCAL* Control. 16, 17, 24, 27, 29

GCP Google Cloud Platform. 17, 31

IaC Infrastructure as Code. 11

K8s Kubernetes. 7
KI Künstliche Intelligenz. 27, 29

NFR nicht-funktionale Anforderung. v, 14, 15, 27, 29
NIST National Institute of Standards and Technology. ii, 10, 16, 31

OCCI Open Cloud Computing Interface. 9
OSCAL Open Security Controls und Assessment Language. ii, v, 9–11, 16, 20, 21, 26–29, 31
OSCO Openshift Compliance Operator. 9, 31
OWL Web Ontology Languge. 6, 8

PoC Proof of concept. ii, v, 3, 5, 14–16, 20–22, 24–27, 29–31
PVP Policy Validation Points. 9, 11, 31

RDF Resource Description Framework. 6

SCAP Security Content Automation Protocol. 10, 31
SDK Software Development Kit. v, 21, 26, 28, 29
SGC Swiss Government Cloud. ii, v, 1, 15, 20
Si001 Si001 - IKT Grundschutz der Bundesverwaltung. v, 1, 2, 4, 11, 22, 24, 27, 29
SSPM System Security Plan Model. 11, 21–24, 27

T-Shirt-Size T-Shirt-Size S, M, L, XL. 14
TOSCA Topology and Orchestration Specification for Cloud Applications. 6–9

UC Use-Case. 14, 15, 27, 30

VM virtual Machine. 6, 8, 9, 11

Eigenständigkeitserklärung

Ich (wir) erkläre(n) hiermit, dass ich (wir) den vorliegenden Leistungsnachweis selber und selbständig verfasst habe(n),

- dass ich (wir) sämtliche nicht von mir (uns) selber stammenden Textstellen und anderen Quellen wie Bilder etc. gemäss gängigen wissenschaftlichen Zitierregeln[23] korrekt zitiert und die verwendeten Quellen klar sichtbar ausgewiesen habe(n);
- dass ich (wir) in einer Fussnote oder einem Hilfsmittelverzeichnis alle verwendeten Hilfsmittel (KI-Assistenzsysteme wie Chatbots[24], Übersetzungs-[25] Paraphrasier-[26] oder Programmierapplikationen[27]) deklariert und ihre Verwendung bei den entsprechenden Textstellen angegeben habe(n);
- dass ich (wir) sämtliche immateriellen Rechte an von mir (uns) allfällig verwendeten Materialien wie Bilder oder Grafiken erworben habe(n) oder dass diese Materialien von mir (uns) selbst erstellt wurde(n);
- dass das Thema, die Arbeit oder Teile davon nicht bei einem Leistungsnachweis eines anderen Moduls verwendet wurden, sofern dies nicht ausdrücklich mit der Dozentin oder dem Dozenten im Voraus vereinbart wurde und in der Arbeit ausgewiesen wird;
- dass ich mir (wir uns) bewusst bin (sind), dass meine (unsere) Arbeit auf Plagiate und auf Drittautorschaft menschlichen oder technischen Ursprungs (Künstliche Intelligenz) überprüft werden kann;
- dass ich mir (wir uns) bewusst bin (sind), dass die Hochschule für Technik FHNW einen Verstoss gegen diese Eigenständigkeitserklärung bzw. die ihr zugrundeliegenden Studierendenpflichten der Studien- und Prüfungsordnung der Hochschule für Technik verfolgt und dass daraus disziplinarische Folgen (Verweis oder Ausschluss aus dem Studiengang) resultieren können.

Windisch, 14. August 2025

Name: Frithjof Hoppe

Unterschrift:

Name: Benjamin Dätwyler

Unterschrift:

[23] z.B. APA oder IEEE
[24] z.B. ChatGPT
[25] z.B. Deepl
[26] z.B. Quillbot
[27] z.B. Github Copilot

A Project Definition mit Azure Paths


```
{
  "_id": {
    "$oid": "6895c4295a0c3aa88aef7aec"
  },
  "id": "143b64f3-8642-4653-a358-3a60dca87fa5",
  "name": "poc-resource",
  "aws_paths": [],
  "azure_paths": [
    {
      "id": "4f0587f9-5f31-4adb-9e37-25bea37db434",
      "path": "/subscriptions/55093e67-09be-455b-bee4-802b5ba38768/resourceGroups/ip5-
    ↪ scggov-sample",
      "plattform": "azure",
      "plattform_policy_reference": []
    },
    {
      "id": "b01faa50-088b-4cb8-a813-31232347c96f",
      "path": "/subscriptions/55093e67-09be-455b-bee4-802b5ba38768/resourceGroups/ip5-
    ↪ scggov",
      "plattform": "azure",
      "plattform_policy_reference": [
        "/subscriptions/55093e67-09be-455b-bee4-802b5ba38768/providers/Microsoft.
    ↪ Authorization/policySetDefinitions/ip5sgcgov-b01faa50-088b-4cb8-a813-31232347
    ↪ c96f-t8.1",
        "/subscriptions/55093e67-09be-455b-bee4-802b5ba38768/providers/Microsoft.
    ↪ Authorization/policySetDefinitions/ip5sgcgov-b01faa50-088b-4cb8-a813-31232347
    ↪ c96f-z2.2"
      ]
    }
  ]
}
```


B Component Definition für Azure AKS


```
{
  "component-definition": {
    "uuid": "e46eee83-a457-42d9-9cc4-96b00e64c638",
    "metadata": {
      "title": "Azure Kubernetes Service (AKS) Component Definition",
      "last-modified": "2025-05-21T13:23:00.000000-00:00",
      "version": "1.0",
      "oscal-version": "1.1.0",
      "parties": [
        {
          "uuid": "ee47836c-877c-4007-bbf3-c9d9bd805a9a",
          "name": "Azure",
          "type": "organization"
        }
      ]
    },
    "components": [
      {
        "uuid": "190eb8f2-0bd0-49b1-b378-8a2f072a1419",
        "type": "service",
```

```
    "title": "Azure Kubernetes Service (AKS)",
    "description": "Managed Kubernetes service in Microsoft Azure.",
    "props": [
      {
        "name":"azure-resource-type",
        "value":"Microsoft.ContainerService/managedClusters"
      }
    ],
    "responsible-roles": [
      {
        "role-id": "provider",
        "party-uuids": [
          "ee47836c-877c-4007-bbf3-c9d9bd805a9a"
        ]
      }
    ],
    "control-implementations": [
      {
        "uuid": "6c7e9b15-6d9c-4b61-a8a0-74ae51540efa",
        "source": "https://ip5-poc-webapp.azurewebsites.net/catalogs/si001.json",
        "description": "Si001 - IKT-Grundschutz in der Bundesverwaltung",
        "implemented-requirements": [
          {
            "uuid": "d1016df0-9b5c-4839-86cd-f9c1d113077b",
            "control-id": "t6.1",
            "description": "Account management for AKS cluster access."
          },
          {
            "uuid": "65e30b37-0640-4844-9f42-b2a7ae944bb1",
            "control-id": "t8.1",
            "description": "Cryptographic key management for AKS nodes.",
            "props": [
              {
                "name" : "azure-policy",
                "remarks" : "Local authentication modes disabled only AD",
                "value": "/providers/Microsoft.Authorization/policyDefinitions/993
↪ c2fcd-2b29-49d2-9eb0-df2c3a730c32"
              }
            ]
          },
          {
            "uuid": "65e30b37-0640-4844-9f42-b2a7ae944bb1",
            "control-id": "z2.2",
            "description": "Restricted network access to AKS",
            "props": [
              {
                "name" : "azure-policy",
                "remarks" : "Should only allow specific external IPs",
                "value": "/providers/Microsoft.Authorization/policyDefinitions/
↪ d46c275d-1680-448d-b2ec-e495a3b6cc89"
              },
              {
                "name" : "azure-policy",
                "remarks" : "Cluster is private",
                "value": "/providers/Microsoft.Authorization/policyDefinitions
↪ /040732e8-d947-40b8-95d6-854c95024bf8"
              }
```

```
                ]
              }
            ]
          }
        ]
      }
    ]
  }
}
```


C System Seucrity Plan für Azure Ressource Group mit AKS


```
{
  "_id": {
    "$oid": "6895c42a5a0c3aa88aef7aed"
  },
  "system-security-plan": {
    "uuid": "9cebf0c6-e4d7-4693-96ec-005989c4b762",
    "metadata": {
      "title": "poc-resource",
      "published": "2025-08-08T09:32:26+00:00",
      "last-modified": "2025-08-08T09:32:26+00:00",
      "version": "0.1",
      "oscal-version": "1.1.3"
    },
    "import-profile": {
      "href": "https://xyz.com"
    },
    "system-characteristics": {
      "system-ids": [
        {
          "id": "143b64f3-8642-4653-a358-3a60dca87fa5"
        }
      ],
      "system-name": "poc-resource",
      "description": "<description>",
      "system-information": {
        "information-types": [
          {
            "uuid": "75af95e1-5aa3-4e9d-ac3e-275dce794ddf",
            "title": "Non-sensitive data",
            "description": "Demo application doesnt hold any sensitive data"
          }
        ]
      },
      "status": {
        "state": "under-development"
      },
      "authorization-boundary": {
        "description": "<authorization-boudary>"
      }
    },
    "system-implementation": {
      "users": [
        {
          "uuid": "b8e99a44-ebd6-4e87-86f1-f8986148a152",
```

```
      "description": "Administers the AKS",
      "role-ids": [
        "admin"
      ]
    }
  ],
  "components": [
    {
      "uuid": "4d5ffa7b-e207-4bc3-b2d4-e1f1bfcec265",
      "type": "service",
      "title": "ip5scggovstorage",
      "description": "ip5scggovstorage",
      "props": [
        {
          "name": "azure-region",
          "value": "switzerlandnorth"
        },
        {
          "name": "azure-resource-type",
          "value": "Microsoft.Storage/storageAccounts"
        },
        {
          "name": "azure-resource-id",
          "value": "/subscriptions/55093e67-09be-455b-bee4-802b5ba38768/
↪ resourceGroups/ip5-scggov-sample/providers/Microsoft.Storage/storageAccounts/
↪ ip5scggovstorage"
        },
        {
          "name": "cac-project-serach-id",
          "value": "4f0587f9-5f31-4adb-9e37-25bea37db434"
        },
        {
          "name": "cac-plattform-type",
          "value": "azure"
        }
      ],
      "status": {
        "state": "state-of-resource"
      }
    },
    {
      "uuid": "2ca44c9c-ed96-44b3-a571-e3236f267b86",
      "type": "service",
      "title": "ip5-poc-app",
      "description": "ip5-poc-app",
      "props": [
        {
          "name": "azure-region",
          "value": "northeurope"
        },
        {
          "name": "azure-resource-type",
          "value": "Microsoft.App/containerApps"
        },
        {
          "name": "azure-resource-id",
```

```
          "value": "/subscriptions/55093e67-09be-455b-bee4-802b5ba38768/
↪ resourceGroups/ip5-scggov/providers/Microsoft.App/containerApps/ip5-poc-app"
        },
        {
          "name": "cac-project-serach-id",
          "value": "b01faa50-088b-4cb8-a813-31232347c96f"
        },
        {
          "name": "cac-plattform-type",
          "value": "azure"
        }
      ],
      "status": {
        "state": "state-of-resource"
      }
    },
    {
      "uuid": "3531f7d0-3963-4db0-91bd-a70f082f6e66",
      "type": "service",
      "title": "ip5-poc-db",
      "description": "ip5-poc-db",
      "props": [
        {
          "name": "azure-region",
          "value": "northeurope"
        },
        {
          "name": "azure-resource-type",
          "value": "Microsoft.App/containerApps"
        },
        {
          "name": "azure-resource-id",
          "value": "/subscriptions/55093e67-09be-455b-bee4-802b5ba38768/
↪ resourceGroups/ip5-scggov/providers/Microsoft.App/containerApps/ip5-poc-db"
        },
        {
          "name": "cac-project-serach-id",
          "value": "b01faa50-088b-4cb8-a813-31232347c96f"
        },
        {
          "name": "cac-plattform-type",
          "value": "azure"
        }
      ],
      "status": {
        "state": "state-of-resource"
      }
    },
    {
      "uuid": "5895e1c7-86a6-48cc-831b-f2fad941e48f",
      "type": "service",
      "title": "ip5-poc-app-Environment",
      "description": "ip5-poc-app-Environment",
      "props": [
        {
          "name": "azure-region",
          "value": "northeurope"
```

```
        },
        {
          "name": "azure-resource-type",
          "value": "Microsoft.App/managedEnvironments"
        },
        {
          "name": "azure-resource-id",
          "value": "/subscriptions/55093e67-09be-455b-bee4-802b5ba38768/
↪ resourceGroups/ip5-scggov/providers/Microsoft.App/managedEnvironments/ip5-poc-
↪ app-Environment"
        },
        {
          "name": "cac-project-serach-id",
          "value": "b01faa50-088b-4cb8-a813-31232347c96f"
        },
        {
          "name": "cac-plattform-type",
          "value": "azure"
        }
      ],
      "status": {
        "state": "state-of-resource"
      }
    },
    {
      "uuid": "5ad01be6-5252-4822-b97b-24b5622f2535",
      "type": "service",
      "title": "Azure Kubernetes Service (AKS)",
      "description": "Managed Kubernetes service in Microsoft Azure.",
      "props": [
        {
          "name": "azure-region",
          "value": "northeurope"
        },
        {
          "name": "azure-resource-type",
          "value": "Microsoft.ContainerService/managedClusters"
        },
        {
          "name": "azure-resource-id",
          "value": "/subscriptions/55093e67-09be-455b-bee4-802b5ba38768/
↪ resourceGroups/ip5-scggov/providers/Microsoft.ContainerService/managedClusters/
↪ sgc-cloudcompliance"
        },
        {
          "name": "oscal-derived-component-definition-uuid",
          "value": "e46eee83-a457-42d9-9cc4-96b00e64c638"
        },
        {
          "name": "oscal-derived-component-uuid",
          "value": "190eb8f2-0bd0-49b1-b378-8a2f072a1419"
        },
        {
          "name": "cac-project-serach-id",
          "value": "b01faa50-088b-4cb8-a813-31232347c96f"
        },
        {
```

```
          "name": "cac-plattform-type",
          "value": "azure"
        }
      ],
      "status": {
        "state": "state-of-resource"
      }
    }
  ]
},
"control-implementation": {
  "description": "Controls implemented by project poc-resource",
  "implemented-requirements": [
    {
      "uuid": "0cf73345-ddd8-43ad-a6c3-80808eb98a12",
      "control-id": "t6.1",
      "by-components": [
        {
          "component-uuid": "5ad01be6-5252-4822-b97b-24b5622f2535",
          "uuid": "f9fc428a-f8df-4a83-9b7e-8698b7dde5e3",
          "description": "t6.1 is implemented by /subscriptions/55093e67-09be-455b
↪ -bee4-802b5ba38768/resourceGroups/ip5-scggov/providers/Microsoft.
↪ ContainerService/managedClusters/sgc-cloudcompliance"
        }
      ]
    },
    {
      "uuid": "7cb35806-c7c3-4998-9f6a-04371ad6a733",
      "control-id": "t8.1",
      "props": [
        {
          "name": "azure-policy",
          "value": "/providers/Microsoft.Authorization/policyDefinitions/993c2fcd
↪ -2b29-49d2-9eb0-df2c3a730c32",
          "remarks": "Local authentication modes disabled only AD"
        }
      ],
      "by-components": [
        {
          "component-uuid": "5ad01be6-5252-4822-b97b-24b5622f2535",
          "uuid": "ebb1c986-aafe-44d8-b8f1-cf99e4e2f9ab",
          "description": "t8.1 is implemented by /subscriptions/55093e67-09be-455b
↪ -bee4-802b5ba38768/resourceGroups/ip5-scggov/providers/Microsoft.
↪ ContainerService/managedClusters/sgc-cloudcompliance"
        }
      ]
    },
    {
      "uuid": "af45aee4-2d58-469c-ad4f-7654979493b3",
      "control-id": "z2.2",
      "props": [
        {
          "name": "azure-policy",
          "value": "/providers/Microsoft.Authorization/policyDefinitions/d46c275d
↪ -1680-448d-b2ec-e495a3b6cc89",
          "remarks": "Should only allow specific external IPs"
        },
```

```
            {
              "name": "azure-policy",
              "value": "/providers/Microsoft.Authorization/policyDefinitions/040732e8-
    ↪ d947-40b8-95d6-854c95024bf8",
              "remarks": "Cluster is private"
            }
          ],
          "by-components": [
            {
              "component-uuid": "5ad01be6-5252-4822-b97b-24b5622f2535",
              "uuid": "1c178a9c-38e0-44e6-b594-f2c1a6075796",
              "description": "z2.2 is implemented by /subscriptions/55093e67-09be-455b
    ↪ -bee4-802b5ba38768/resourceGroups/ip5-scggov/providers/Microsoft.
    ↪ ContainerService/managedClusters/sgc-cloudcompliance"
            }
          ]
        }
      ]
    }
  }
}
```


D Assessment Plan Results


```
{
  "_id": {
    "$oid": "6895c5b265d105a3d7246194"
  },
  "assessment-results": {
    "uuid": "7a5d74ba-392b-4e9e-bbfc-0ffd2cadd393",
    "metadata": {
      "title": "Assesment plan for project 143b64f3-8642-4653-a358-3a60dca87fa5",
      "last-modified": "2025-08-08T09:38:58.738681+00:00",
      "version": "1.0",
      "oscal-version": "1.1.3"
    },
    "import-ap": {
      "href": "43064910-136e-47fd-9cda-90f57f383e69"
    },
    "results": [
      {
        "uuid": "43e7a4bd-4491-47a6-8a47-b152eb90964f",
        "title": "Assessment for project 143b64f3-8642-4653-a358-3a60dca87fa5",
        "description": "Assessment for project 143b64f3-8642-4653-a358-3a60dca87fa5",
        "start": "2025-08-08T09:38:58.737161+00:00",
        "end": "2025-08-08T09:38:58.737163+00:00",
        "reviewed-controls": {
          "control-selections": [
            {
              "include-controls": [
                {
                  "control-id": "t8.1"
                },
                {
                  "control-id": "z2.2"
                },
```

```
            {
              "control-id": "z2.2"
            }
          ]
        }
      ]
    },
    "assessment-log": {
      "entries": [
        {
          "uuid": "c76bbf2e-522e-4799-953c-8de9dddb3d3c",
          "title": "Assignt azure policy initiative",
          "description": "Assigned initaitve assignment-ip5sgcgov-b01faa50-088b-4
↪ cb8-a813-31232347c96f-t8.1 to scope /subscriptions/55093e67-09be-455b-bee4-802
↪ b5ba38768/resourceGroups/ip5-scggov",
          "start": "2025-08-08T09:38:55.469604+00:00",
          "end": "2025-08-08T09:38:55.469606+00:00"
        },
        {
          "uuid": "613419e0-89da-4e6d-a9a2-9b9094f29460",
          "title": "Assignt azure policy initiative",
          "description": "Assigned initaitve assignment-ip5sgcgov-b01faa50-088b-4
↪ cb8-a813-31232347c96f-z2.2 to scope /subscriptions/55093e67-09be-455b-bee4-802
↪ b5ba38768/resourceGroups/ip5-scggov",
          "start": "2025-08-08T09:38:56.098313+00:00",
          "end": "2025-08-08T09:38:56.098318+00:00"
        },
        {
          "uuid": "81122e29-1f85-4593-9b59-72472baa5c81",
          "title": "Analyzed ressource 5ad01be6-5252-4822-b97b-24b5622f2535 is
↪ compliant",
          "description": "Ressource with id /subscriptions/55093e67-09be-455b-bee4
↪ -802b5ba38768/resourceGroups/ip5-scggov/providers/Microsoft.Authorization/
↪ policyAssignments/assignment-ip5sgcgov-b01faa50-088b-4cb8-a813-31232347c96f-t8
↪ .1 is compliant to policy /subscriptions/55093e67-09be-455b-bee4-802b5ba38768/
↪ resourceGroups/ip5-scggov/providers/Microsoft.Authorization/policyAssignments/
↪ assignment-ip5sgcgov-b01faa50-088b-4cb8-a813-31232347c96f-t8.1",
          "start": "2025-08-08T09:38:57.036663+00:00",
          "end": "2025-08-08T09:38:57.036669+00:00"
        },
        {
          "uuid": "03c95229-b8d4-466b-a05e-d8ee7a0d4a77",
          "title": "Analyzed ressource 5ad01be6-5252-4822-b97b-24b5622f2535 is
↪ compliant",
          "description": "Ressource with id /subscriptions/55093e67-09be-455b-bee4
↪ -802b5ba38768/resourceGroups/ip5-scggov/providers/Microsoft.Authorization/
↪ policyAssignments/assignment-ip5sgcgov-b01faa50-088b-4cb8-a813-31232347c96f-z2
↪ .2 is compliant to policy /subscriptions/55093e67-09be-455b-bee4-802b5ba38768/
↪ resourceGroups/ip5-scggov/providers/Microsoft.Authorization/policyAssignments/
↪ assignment-ip5sgcgov-b01faa50-088b-4cb8-a813-31232347c96f-z2.2",
          "start": "2025-08-08T09:38:58.018284+00:00",
          "end": "2025-08-08T09:38:58.018287+00:00"
        },
        {
          "uuid": "1353449d-1e92-4d47-ac60-98f5ee9bb98d",
          "title": "Analyzed ressource 5ad01be6-5252-4822-b97b-24b5622f2535 is
↪ compliant",
```

```
          "description": "Ressource with id /subscriptions/55093e67-09be-455b-bee4
↪ -802b5ba38768/resourceGroups/ip5-scggov/providers/Microsoft.Authorization/
↪ policyAssignments/assignment-ip5sgcgov-b01faa50-088b-4cb8-a813-31232347c96f-z2
↪ .2 is compliant to policy /subscriptions/55093e67-09be-455b-bee4-802b5ba38768/
↪ resourceGroups/ip5-scggov/providers/Microsoft.Authorization/policyAssignments/
↪ assignment-ip5sgcgov-b01faa50-088b-4cb8-a813-31232347c96f-z2.2",
          "start": "2025-08-08T09:38:58.736687+00:00",
          "end": "2025-08-08T09:38:58.736690+00:00"
        }
      ]
    },
    "findings": [
      {
        "uuid": "215dea29-1aa0-4b2b-ad7f-ff8b0295474d",
        "title": "Control 5ad01be6-5252-4822-b97b-24b5622f2535 is 'compliant' to
↪ control_id t8.1 in reguards of policy /providers/Microsoft.Authorization/
↪ policyDefinitions/993c2fcd-2b29-49d2-9eb0-df2c3a730c32",
        "description": "Control 5ad01be6-5252-4822-b97b-24b5622f2535 is 'compliant
↪ ' to control_id t8.1 in reguards of policy /providers/Microsoft.Authorization/
↪ policyDefinitions/993c2fcd-2b29-49d2-9eb0-df2c3a730c32",
        "props": [
          {
            "name": "compliant",
            "value": "non-compliant"
          },
          {
            "name": "oscal-component-id",
            "value": "5ad01be6-5252-4822-b97b-24b5622f2535"
          }
        ],
        "target": {
          "type": "oscal-component-id",
          "target-id": "component-5ad01be6-5252-4822-b97b-24b5622f2535",
          "status": {
            "state": "non-compliant"
          }
        }
      },
      {
        "uuid": "5d150b2a-ae93-43da-b7d6-2f63369f9799",
        "title": "Control 5ad01be6-5252-4822-b97b-24b5622f2535 is 'compliant' to
↪ control_id z2.2 in reguards of policy /providers/Microsoft.Authorization/
↪ policyDefinitions/d46c275d-1680-448d-b2ec-e495a3b6cc89",
        "description": "Control 5ad01be6-5252-4822-b97b-24b5622f2535 is 'compliant
↪ ' to control_id z2.2 in reguards of policy /providers/Microsoft.Authorization/
↪ policyDefinitions/d46c275d-1680-448d-b2ec-e495a3b6cc89",
        "props": [
          {
            "name": "compliant",
            "value": "non-compliant"
          },
          {
            "name": "oscal-component-id",
            "value": "5ad01be6-5252-4822-b97b-24b5622f2535"
          }
        ],
        "target": {
```

```
            "type": "oscal-component-id",
            "target-id": "component-5ad01be6-5252-4822-b97b-24b5622f2535",
            "status": {
              "state": "non-compliant"
            }
          }
        },
        {
          "uuid": "b8518658-0585-4f37-a51b-4aa21715ec3d",
          "title": "Control 5ad01be6-5252-4822-b97b-24b5622f2535 is 'compliant' to
    ↪ control_id z2.2 in reguards of policy /providers/Microsoft.Authorization/
    ↪ policyDefinitions/040732e8-d947-40b8-95d6-854c95024bf8",
          "description": "Control 5ad01be6-5252-4822-b97b-24b5622f2535 is 'compliant
    ↪ ' to control_id z2.2 in reguards of policy /providers/Microsoft.Authorization/
    ↪ policyDefinitions/040732e8-d947-40b8-95d6-854c95024bf8",
          "props": [
            {
              "name": "compliant",
              "value": "non-compliant"
            },
            {
              "name": "oscal-component-id",
              "value": "5ad01be6-5252-4822-b97b-24b5622f2535"
            }
          ],
          "target": {
            "type": "oscal-component-id",
            "target-id": "component-5ad01be6-5252-4822-b97b-24b5622f2535",
            "status": {
              "state": "non-compliant"
            }
          }
        }
      ]
    }
  ]
}
}
```